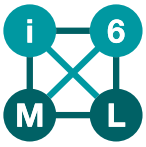


Chair of Machine Learning and Reasoning (Computer Science 6)

RWTH AACHEN UNIVERSITY

This thesis was submitted to the Chair of Machine Learning and Reasoning.

Semantic Partitioning for Partially Observable Deterministic Task and Motion Planning

Master Thesis

Presented by

Hani Alassiri Alhabboub
422433

Supervised by Prof. Hector Geffner, Ph.D.

1st Examiner Prof. Hector Geffner, Ph.D.

2nd Examiner Prof. Christopher Morris

Aachen, June 10, 2026

Abstract

This thesis extends classical Task and Motion Planning (TAMP) to partial observability by integrating a Semantic Boxel representation—task-relevant cuboids that discretise only the objects and the regions they occlude—into a PDDLStream-based planner. Compiling the belief over Boxel occupancy to Know-If literals lets a classical planner reason about hidden objects and plan sensing actions in the same search, without enumerating an exhaustive state space. On three tabletop tasks evaluated against a uniform-grid baseline at the same cell scale, the adaptive partition produces an order of magnitude fewer cells, lowers per-call planning time accordingly, and solves find-and-tray-stack scenes that the uniform baseline effectively cannot (39.8 % vs 1.3 % success). On the `holding` task—the closest analogue to TAMPURA's hidden-object *Find Die* setting—our planner takes a mean per-episode wall-clock of 13.7 s against TAMPURA's reported 57 s, but succeeds less often (42 % vs an inferred ≥ 63 %). The contrast is architectural—our deterministic, replanning planner against TAMPURA's learned probabilistic model—and implies no speed or quality winner.

Contents

1 Introduction

We want robots that can carry out everyday tasks on their own—fetching an object, clearing a table—tasks that demand both multi-step reasoning and careful physical manipulation. Task and Motion Planning (TAMP) studies how to do exactly this [7, 10]. TAMP connects a high-level goal (e.g., ”fetch the blue cup from the kitchen”) to the actual motions the robot must perform to achieve it. TAMP combines discrete task planning with continuous motion planning: symbolic planners generate high-level sequences of abstract actions—what is known as a task plan. However, symbolic planning alone is insufficient for real-world robotic execution. Because the symbolic plan ignores geometry, it can call for actions the robot cannot physically carry out. Specifically, we must determine whether the proposed symbolic actions are feasible within the robot's continuous geometric and kinematic constraints, and if so, identify the concrete parameters required to execute them—such as object poses, grasp configurations, and collision-free trajectories. For instance, a symbolic instruction like (*pick A*) is only viable if object A is actually reachable, graspable, and a feasible motion exists to carry out the action.

One widely used framework for this is PDDLStream [10]. PDDLStream extends the classical Planning Domain Definition Language (PDDL) [23] by incorporating streams—lazily-evaluated, declarative procedures that can interface with external solvers. These streams are capable of generating or validating continuous parameters on demand, allowing the symbolic planner to retrieve geometric or kinematic information only as needed. This simplifies the coupling between high-level symbolic reasoning (what to do) and low-level geometric feasibility (how to do it), making the planning process both flexible and efficient.

However, the original PDDLStream framework assumes full observability and deterministic action outcomes—assumptions that rarely hold in practice. In practice, robots frequently operate under partial observability, due to occlusions, limited sensor coverage, or dynamically changing environments. In such settings, crucial information—such as the location of a target object or the presence of an obstacle—may be unknown at the start, so the robot has to actively look for it while carrying out the task (Figure 1.1).

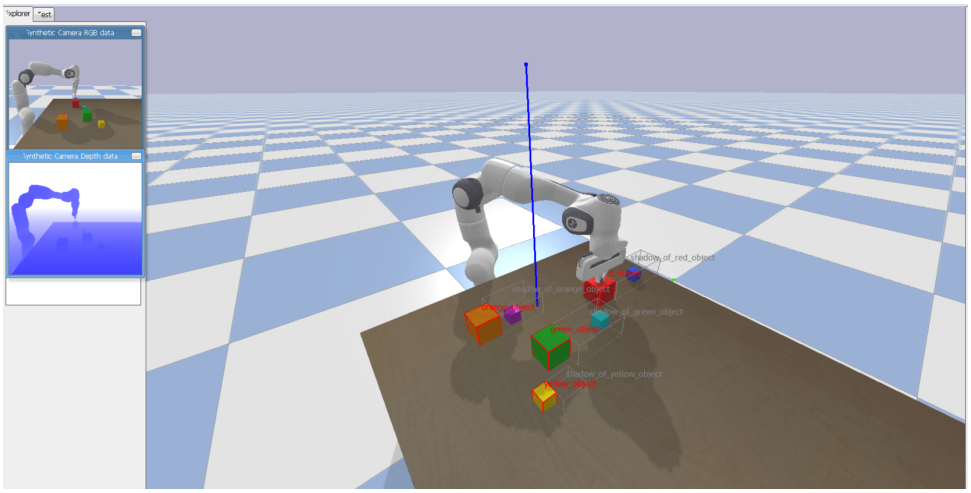


Figure 1.1: A 7-DOF Franka Panda at a cluttered tabletop, where the target object may be hidden behind others and must be located before it can be retrieved. The corner panels are the fixed overhead camera's synthetic RGB (top) and depth (bottom) views.

Addressing this limitation is the central focus of this thesis: extending PDDLStream to operate under partial observability. This includes the ability to represent and reason over belief states that capture uncertainty in continuous spatial variables. Naive representations—such as a uniform voxel grid over 3D space—make the belief space blow up in size and quickly become impractical to plan with. A useful belief representation must therefore be detailed enough to be expressive, yet small enough to plan with quickly. The core research question is: *How can we formally define and integrate a belief representation based on semantic partitioning into a POD-TAMP framework?* The stance of this thesis is simple: *space is part of the planning problem*. Where a hidden object might be, and what blocks the view of it, are not details to be collapsed into a probabilistic occupancy grid but structure the planner should weigh directly. Here, we present a novel approach to discretizing the workspace around the objects the robot detects and the regions they occlude. The unit of this discretization is the Semantic Boxel: a task-relevant, occlusion-aware cuboid, built from objects' known 3D models, that the robot uses to represent its belief. Unlike a dense, uniform discretization, this provides a structured, scalable way to represent and reason about spatial uncertainty.

This thesis develops a system in which a robot, facing uncertainty about object locations, can plan and execute a sequence of information-gathering and manipulation actions. It reasons over abstract beliefs about which Boxels objects might occupy, using PDDLStream to connect this reasoning to continuous motion and manipulation. Sensing is a symbolic action evaluated against a fixed overhead camera, not a viewpoint the robot moves to; and because the underlying planner is classical, partial observability is handled by optimistic sensing with reactive replanning rather than contingent planning—a sensing action is assumed to succeed, and the system replans if execution shows otherwise. The result is a framework that makes

planning under spatial uncertainty simpler and more reliable. The approach is demonstrated on tabletop hidden-object retrieval and stacking; the partial-observability mechanism is the contribution, and longer task horizons are a natural extension.

1.1 Contributions of this thesis

The central contribution is a *representation*: we make occlusion first-class symbolic planning state. Each volume the robot cannot see becomes a named region that a classical, partially observable deterministic (POD) planner reasons over and resolves by sensing, rather than a sub-symbolic spatial belief such as the dense visibility voxel grid of TAMPURA's *Find Die* benchmark, which feeds a learned model of action outcomes [7]. The three components below deliver this representation; the evaluation correspondingly isolates it by ablation (the same planner on the adaptive partition against a uniform grid), alongside an architectural comparison with TAMPURA, rather than claiming a benchmark win, and is conducted under oracle perception (Section 6.6).

- The *Semantic Boxel* abstraction: an occlusion-aware, object-centric partitioning of the workspace that discretizes only the task-relevant regions—the objects themselves and the volumes they occlude—rather than partitioning all space uniformly.
- A Semantic POD-TAMP model that uses Know-If Fluents over Boxels as a compact belief representation, letting a classical PDDL planner reason about partial observability via optimistic sensing with reactive replanning.
- A PDDLStream realization of this model that combines the discrete Boxel belief with stream-certified continuous geometry, so that information-gathering and manipulation actions interleave within a single planning domain.

1.2 Thesis outline

Chapter 2 provides essential background on classical planning, reasoning under uncertainty, and the PDDLStream framework that this work builds upon. Chapter 3 reviews related work in TAMP under partial observability, belief representation, and adjacent planning paradigms. Chapter 4 details the methodology, including the formalization of Boxels through adaptive semantic discretization, the Semantic POD-TAMP model, and the PDDLStream integration. Chapter 5 presents the experimental setup, the metrics and baselines, and the headline evaluation results. Chapter 6 interprets those results and consolidates the accepted simplifications. Chapter 7 summarizes the contributions and outlines directions for future work.

2 Background

This chapter builds up the pieces the thesis depends on, ending with PDDLStream and how 3D space is represented for planning.

2.1 AI Planning Fundamentals

Artificial Intelligence (AI) planning produces a sequence of actions—a plan—that gets from a given initial state to a specified goal [13].

2.1.1 State-Space Models in Classical Planning

The formal basis for classical planning problems is a *state-space model* [12, 13, 17]. A state-space model defines every possible configuration of the world and how actions move between configurations.

More formally, a deterministic state-space model can be described as a tuple $\Pi = \langle S, s_0, S_G, Act, A, f \rangle$, where:

- S: A set representing all possible *states* the world can be in. For example, in a simple blocksworld problem with two blocks A and B and a table, a state could be "Block A is on Block B, and Block B is on the table, and the robot hand is empty."
- $s_0 \in S$: The specific *initial state* from which planning begins. E.g., "Block A is on the table, Block B is on the table, hand empty."
- $S_G \subseteq S$: The set of *goal states*. Any state in this set satisfies the desired objective. E.g., any state where "Block A is on Block B."
- Act: A set of all possible *actions* the agent can perform. E.g., *pick blockX*, *stack blockX on blockY*.
- $A(s) \subseteq Act$: A function that, for any given state $s \in S$, returns the subset of actions from Act that are *applicable* (can be executed) in state s. For example, *pick blockA* is applicable only if Block A is clear (nothing on top of it) and the robot hand is empty.
- $f(a, s) \rightarrow s'$: A deterministic *state transition function*. Given a state s and an applicable action $a \in A(s)$, $f(a, s)$ returns the unique successor state s' that results from executing action a in state s. For example, if s is "A on table, B on table, hand empty" and a is *pick blockA*, then $s' = f(a, s)$ would be "B on table, hand holding A."

A *solution* or *plan* is then a sequence of actions $\sigma = [a_0, a_1, \dots, a_n]$ such that if s_i is the state before action a_i, then $a_i \in A(s_i)$, $s_{i+1} = f(a_i, s_i)$, and the final state s_{n+1} is in S_G. Often, we

are interested in an *optimal plan*, which could be the shortest sequence of actions or one that minimizes some associated cost.

Compact Representation: STRIPS, and PDDL Explicitly enumerating all possible states S is infeasible for all but the simplest problems. Therefore, planning problems are typically described in a more compact, *factored* representation using a planning language. In this representation, a state is defined by a set of propositions (or ground atoms) that are currently true.

A foundational language for this is **STRIPS (Stanford Research Institute Problem Solver)** [8]. In STRIPS:

- **States** are represented as sets of true propositions (e.g., `(on A B)`, `(ontable B)`, `(handempty)`).
- **Actions** are defined by:
  - *Preconditions*: A set of propositions that must be true in the current state for the action to be applicable.
  - *Effects*: Specified as two lists:
    - *Add-list*: A set of propositions that become true after the action is executed.
    - *Delete-list*: A set of propositions that become false (are no longer true) after the action.
- The transition function $f(a, s)$ is implicitly defined: the new state s' is obtained by taking state s, removing all propositions in the delete-list of a, and then adding all propositions in the add-list of a.
- The initial state (s_0) is a set of initially true propositions, and the goal (S_G) is specified by a set of propositions that must be true in any goal state.

The **Planning Domain Definition Language (PDDL)** [23] builds upon and extends the STRIPS representation, providing a more expressive and standardized syntax for a wider range of classical planning problems. In PDDL:

- **Domain File** defines:
  - *Types*: Categories for objects (e.g., `block`, `robot`).
  - *Predicates*: Relations or properties (e.g., `(on ?x - block ?y - block)`).
  - *Action Schemas*: Templates for actions, generalizing STRIPS operators. For instance:

```
1  (:action stack
2      :parameters (?obj1 - block ?obj2 - block)
3      :precondition (and (holding ?obj1) (clear ?obj2))
4      :effect (and (on ?obj1 ?obj2) (not (holding ?obj1))
5                   (not (clear ?obj2)) (handempty) (clear
                        ?obj1))
6  )
```

In STRIPS terms, this action's delete-list contains `(holding ?obj1)` and `(clear ?obj2)`, while its add-list contains `(on ?obj1 ?obj2)`, `(handempty)`, and `(clear ?obj1)`.

- **Problem File** specifies:
  - *Objects*: Specific objects (e.g., `A, B, C - block`).
  - *Initial State (`Init`)*: Ground atoms true at the start.
  - *Goal (`G`)*: Ground atoms that must be true in a goal state.

A PDDL problem $P = \langle \text{Domain}, \text{Instance} \rangle$ thus compactly encodes the state model $S(P)$. Ground actions (action schemas with parameters replaced by specific objects) form the set Act. More advanced PDDL versions also support features beyond STRIPS, such as conditional effects (`(when <condition> <effect>)`), numeric fluents, and axioms (derived predicates).

2.2 Planning under Partial Observability

Classical planning's assumption of full observability is often violated in robotics. When an agent has incomplete information about the true state of the world, it must reason about its *belief state*.

2.2.1 Belief States and K-Literals

A belief state represents the agent's current set of possible world states consistent with its history of actions and observations. We refer to the setting this thesis targets as *Partially Observable Deterministic (POD)* planning: the world itself behaves deterministically and the only uncertainty is the agent's incomplete knowledge of the initial situation, which sensing resolves. "POD" is a descriptive label we adopt for this setting rather than established terminology in the contingent-planning literature it builds on [1, 5, 12]. The belief state b is explicitly a set of possible states. Actions transition this set, and observations prune it. A common way to reason about knowledge in POD planning is through K-literals [4, 12]. K-literals make the robot's knowledge about a proposition p explicit. For example:

- $K(p)$ signifies "p is known to be true."
- $K(\neg p)$ signifies "p is known to be false."

If neither $K(p)$ nor $K(\neg p)$ holds, the truth of p is unknown. Separately, p is "possibly true" whenever $K(\neg p)$ does not hold—which also covers the case where p is known to be true.

K-literals are the basis of a *translation* that compiles a partially observable problem into a classical one [12]. Each literal L of the original problem is replaced by the two fluents $K(L)$ and $K(\neg L)$; in the translated problem these are made true in the initial state only where the agent actually knows the corresponding value, and a literal whose value is unknown contributes neither. The translated problem therefore carries no uncertainty in its initial state and can be

solved by an ordinary classical planner, which now searches for a plan that reaches a desired *state of knowledge*—for example K(goal_achieved)—rather than a desired world state. Because the pair $K(L)$ and $K(\neg L)$ together encodes whether the agent knows the truth of L, we call such a knowledge fluent a *Know-If Fluent (KIF)*. KIFs are the belief representation used throughout this thesis (Chapter 4).

2.2.2 Probabilistic vs. Deterministic Partial Observability

POD planning differs from planning with Partially Observable Markov Decision Processes (POMDPs) [19]—a distinction that matters because we later compare our approach to POMDP-based TAMP. In POMDPs, the world is modeled as a Markov Decision Process (MDP) with probabilistic transitions and observations, and the belief state is a probability distribution over world states. While highly expressive, POMDPs are computationally very demanding. POD planning assumes outcomes and observations are deterministic but unknown. This makes it more tractable than POMDPs, and it fits robotic problems where the uncertainty comes from not knowing the world rather than from actions behaving randomly.

2.3 Task and Motion Planning (TAMP)

Task and Motion Planning (TAMP) connects high-level task goals with a robot's actual physical movements [9]. The core problem is ensuring that a plan is physically possible. High-level planners often ignore geometry, leading to plans that seem correct but can't be executed. Yet, including all geometric details from the start makes the problem too complex to solve. Combining high-level logic with real-world geometry is the central difficulty of TAMP, and frameworks like PDDLStream exist to bridge it.

2.3.1 PDDLStream for TAMP

PDDLStream [10] is a TAMP framework built to bridge this symbolic-continuous gap. It extends the classical Planning Domain Definition Language (PDDL) by introducing the core concept of **streams**.

Streams: Declarative Procedures for Continuous Parameters A stream in PDDLStream is a procedure that generates or tests values for continuous parameters required by symbolic actions. These parameters might include object poses, robot configurations, grasp transformations, or motion trajectories. Streams have both a procedural and a declarative component:

- **Procedural Component (Conditional Generator):** This is the actual code (e.g., a Python function) that, given some input values (which can be discrete symbols or other continuous values), produces a (potentially infinite) sequence of output values. For example, an inverse kinematics (IK) stream, given an object, its target pose, and a grasp,

would generate a sequence of robot configurations that achieve that end-effector pose. A collision-checking stream, given a trajectory, would test if it's collision-free.

- **Declarative Component (Stream Specification):** This part is described in a PDDL-like syntax and informs the symbolic planner what the stream does, what inputs it requires, what outputs it produces, and, crucially, what logical conditions (fluents) are associated with these inputs and outputs. This specification includes:
  - `:inputs`: The typed parameters the stream procedure takes.
  - `:domain`: A set of PDDL facts (preconditions) that must be true for the stream to be applicable for a given set of inputs. For example, an IK stream might require that a valid pose and grasp for the object are already known.
  - `:outputs`: The typed parameters that the stream procedure generates.
  - `:certified`: A set of PDDL facts that are guaranteed to be true if the stream successfully produces a set of outputs for given inputs. These "certified fluents" are added to the planner's state and represent new knowledge derived from the continuous domain. For instance, if an IK stream successfully finds a configuration `?q` for picking object `?obj` at pose `?p` with grasp `?g`, it might certify the fluent `(KinSolution ?obj ?p ?g ?q)`.

PDDLStream employs a **lazy evaluation** strategy. Streams are not called exhaustively beforehand; instead, the symbolic planner calls them "on-demand" when it encounters an action whose preconditions require a continuous parameter to be instantiated or a continuous condition to be tested.

Example PDDLStream Workflow Consider a symbolic action `(pick ?obj ?p ?g ?q ?t)` (pick object `?obj` from pose `?p` using grasp `?g` via configuration `?q` along trajectory `?t`). For this action to be applicable:

1. A stream might first be called to sample a stable `?p` for `?obj` on a surface. If successful, it certifies `(AtPose ?obj ?p)`.
2. Then, another stream samples a grasp `?g` for `?obj`. If successful, it certifies `(Grasp ?obj ?g)`.
3. An IK stream then takes `?obj, ?p, ?g` as input and tries to find a robot configuration `?q`. If successful, it certifies `(KinSolution ?obj ?p ?g ?q)`.
4. Finally, a motion planning stream takes the current configuration and `?q` as input to find a collision-free trajectory `?t`, certifying `(Motion ?q_start ?t ?q)`.

Only if all these streams succeed and certify their respective fluents will the preconditions of the symbolic `(pick ...)` action be met.

The overall PDDLStream algorithm often reduces a TAMP problem to a sequence of finite PDDL problems, iteratively calling streams to discover more certified fluents until a com-

plete, grounded plan is found. The original PDDLStream framework, however, was primarily designed for fully observable and deterministic settings.

2.4 Spatial Representation for Robotic Planning

A robot needs some way to represent the 3D space around it and the objects in it. Representing space is harder under partial observability, because the robot must also represent what it does not yet know.

2.4.1 Discretization of Continuous Space

Robots operate in continuous space, but symbolic planners reason over discrete states — so the continuous space has to be discretized somehow.

Voxel Grids and Octrees A common approach to represent 3D environments is to discretize them into a grid of uniform volumetric pixels, or **voxels**. Each voxel can store information, such as whether it is occupied, free, or unknown. While straightforward, uniform voxel grids can be memory-intensive and computationally expensive for large environments or high resolutions.

Octrees (Figure 2.1) [18] offer a more efficient hierarchical data structure for representing 3D space. An octree is a tree data structure in which each internal (non-leaf) node has exactly eight children, while leaf nodes have none. It starts with a single large voxel encompassing the entire space of interest. If this voxel is not homogeneous (e.g., it contains both free and occupied space), it is recursively subdivided into eight smaller child voxels (octants).

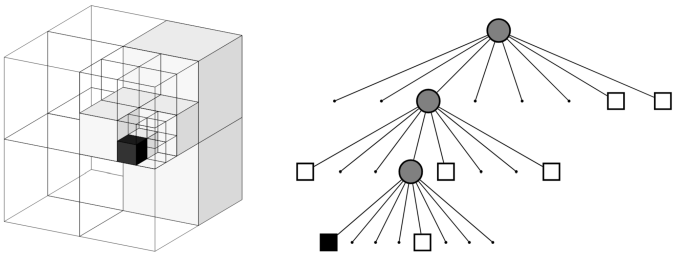


Figure 2.1: Example of an octree storing free (shaded white) and occupied (black) cells. The volumetric model is shown on the left and the corresponding tree representation on the right.

This subdivision continues until voxels are homogeneous or a minimum resolution/maximum depth is reached. This process creates a tree where large, uniform regions (whether free, occupied, or unknown) are represented by single nodes at a shallower depth, making it highly efficient for representing sparse environments in comparison to uniform grid voxelization. The **OctoMap** framework [18] uses this principle to build probabilistic 3D occupancy maps from

noisy sensor data. In such a framework, unobservable space is typically represented implicitly by the absence of nodes in a given volume; querying such a region returns an ”unknown” status. The same octree structure also underpins learning-based 3D methods such as OctNet [27], which runs deep convolutions efficiently on an octree rather than building an occupancy map; here it is the occupancy-mapping use that is relevant.

Sidd's Critical Regions To make planning more efficient, space can be divided based on task-relevant meaning rather than uniform geometric criteria. One such approach is the concept of **Critical Regions (CRs)**, introduced by Molina et al. [24] and later extended by Shah and Srivastava [29]. In the original formulation, a critical region is a part of the configuration space with a low probability of being sampled by a uniform sampler yet essential to a solution—narrow passages such as doorways [24]. Shah and Srivastava generalised this to regions where task-relevant features—controller success rates, object visibility, or transition likelihoods—change significantly [29].

In this line of work the regions are not hand-specified: a deep network is trained to *predict* the critical regions of a given environment from that scene's input, and the predictor generalises to environments it was not trained on. Once predicted, a region-based Voronoi diagram (RBVD) partitions the configuration space around the CRs: each point is assigned to the nearest critical region, forming abstract states. Transitions between these abstract states define abstract actions, forming the backbone for hierarchical planning. By focusing computation on these task-relevant regions, this approach scales planning more effectively, but the regions are predicted once per environment and then held fixed; they do not adapt as objects move within a scene during execution.

Our work combines these techniques, as explained in Chapter 4.

3 Related Work

Integrating task and motion planning with partial observability is challenging, particularly when a robot must act on incomplete information about its environment. This section reviews previous work on this topic, focusing on planning approaches and methods for representing a robot's beliefs about the world.

3.1 TAMP under Partial Observability

Existing approaches to TAMP under partial observability generally either use formal, probabilistic models or more ad-hoc, reactive methods, or a combination of both.

3.1.1 POMDP-based TAMP Solutions

Several works formulate planning under uncertainty as a POMDP to handle it in a principled, risk-aware way [7, 28]. TAMPURA [7], for example, learns a coarse abstract model of each given controller's preconditions and effects, builds a non-deterministic MDP, and solves it with an uncertainty-aware solver (LAO*) [16] to produce risk-aware, information-gathering plans rather than provably optimal ones. Saleem et al. [28], by contrast, address contact-rich *manipulation* under object-pose uncertainty—not full task-and-motion planning—computing partial policies with an anytime heuristic-search POMDP solver. However, a persistent challenge is the representation of belief over continuous properties, particularly object poses. TAMPURA [7] samples object placements in continuous pose space and, in its real-robot pipeline, draws its object-pose belief from a perception front-end, Bayes3D [14]: a generative inverse-graphics model that infers a posterior over 3D scenes by rendering candidate object arrangements and scoring them against the observed depth image.

3.1.2 Partially Grounded Plans in TAMP

In contrast to formal probabilistic models, other TAMP systems handle uncertainty by deferring it to execution [26]. These methods generate partially grounded plans, leaving "gaps" for actions with unknown outcomes. During execution, specialized reactive controllers attempt to fill these gaps. If a controller fails, the failure is treated as a new constraint, and the system replans. This reactive approach avoids the need to model outcome probabilities explicitly. However, because it does not reason about uncertainty proactively, it can struggle with problems that require multi-step, strategic information gathering, such as deciding which of several actions is most likely to reveal a hidden object.

3.1.3 Modern Approaches to TAMP under Uncertainty

Recent works have leveraged learning and large models to address partial observability, offering different strategies than our own.

One approach adds high-level strategic reasoning on top of standard planners to handle real-world messiness. For instance, Ma et al. [22] first reason backward from a vague goal to decide which objects are important (Task-level Backward Search), and then use a constraint graph to find a safe order to move objects in a cluttered scene (Object Manipulation Constraint Graph). Our approach is more fundamental. Instead of adding a separate strategic layer, we introduce a new geometric belief representation, Boxels. This lets our planner treat which regions are observed versus occluded as first-class planning state, so information-gathering is a core planner action rather than a separate, pre-planning step; the visibility signal is currently supplied by an oracle.

Another approach sources the planner's belief from large foundation models. Zhao et al. [30] use a Vision-Language Model (VLM) as an uncertainty estimator: the VLM answers the planner's perceptual queries about an image (for example, "Is the drawer empty?"), and the resulting belief feeds a *symbolic* belief-space planner—a pairing of learned perception with structured reasoning that is reported to outperform end-to-end VLM planners. CoCo-TAMP [21] is similar in spirit, using an LLM's commonsense priors to shape the belief over where task-relevant objects are likely to be, again within a partially observable TAMP loop. The difference from our work is the *source* of the belief: theirs is learned (a VLM or LLM), whereas ours is a structured, object-centric geometric partition into discrete semantic Boxels, with object poses currently supplied by a ground-truth oracle rather than a learned detector. Integrating a real detector is left to future work.

Finally, some methods move away from symbolic planning altogether and instead use end-to-end learning. Bai et al. [2], for example, train a learned policy that selects task-relevant *virtual* camera viewpoints and re-renders the scene from them—rather than physically moving the camera—before acting (Task-Aware Virtual View Exploration, TVVE). This is fundamentally different from our model-based approach, where looking around is not a learned reflex but an explicit action chosen by the planner because its world model indicates that knowledge is missing. Because our method separates planning from execution, it is more transparent and can be retargeted to new goals without retraining a policy; adapting it to a new environment, however, currently requires re-tuning hardcoded geometric constants.

3.1.4 Belief-Space Planning and Replanning

A long line of work plans directly in *belief space*, treating the robot's distribution over world states as the object of planning. Kaelbling and Lozano-Pérez [20] introduced hierarchical planning in belief space for partially observable manipulation, and Hadfield-Menell et al. [15]

gave a modular formulation that interleaves symbolic and geometric reasoning over beliefs. A practical alternative to maintaining an explicit stochastic model is to *determinise and replan*: SS-Replan [11] plans deterministically in a hybrid belief space and replans whenever a new observation arrives. Our system shares this determinise-and-replan strategy—we commit to an optimistic deterministic plan and replan on each observation rather than synthesising a probabilistic policy. What we add is orthogonal to the replanning loop: a spatial belief in which occlusion is explicit, named planning state, so information-gathering is composed *within* a plan rather than emerging only across replans.

3.1.5 Partially Observable Deterministic Planning

A separate tradition handles partial observability *without* probabilities, by reasoning about what is *known*. Partially observable deterministic (POD) planning represents knowledge with explicit *knowledge literals* and reduces the problem to classical planning by compilation. The translation-based contingent planner CLG [1] and the classical-replanning K-replanner [4] established this reduction; LW1 [5] made it scalable with a *linear* translation; and a parallel route compiles partial observability to fully-observable non-deterministic (FOND) planning [25]. The underlying models are treated by Geffner and Bonet [12]. This thesis builds directly on the POD line: we carry knowledge literals—e.g. whether a region's contents are known—inside the PDDL state and let a classical planner reason over them.

To our knowledge, POD planning has not previously been combined with task and motion planning, nor with a geometric belief representation that makes *which regions are occluded* part of the symbolic state. Occlusion-aware object retrieval has been studied—for instance by searching the likely locations of an occluded target drawn from a *learned* distribution [3]—and partially observable TAMP systems increasingly source their belief from learned models [21, 30] or offline probabilistic abstractions [7]; but we are not aware of prior work that represents occluded volumes as named, object-centric symbolic state resolved by sensing within a PDDLStream TAMP loop—the gap this thesis addresses.

3.1.6 Summary and Research Gap

These approaches occupy distinct points in the design space. POMDP-based methods such as TAMPURA [7] reason about uncertainty proactively but pay for offline model-learning, and they keep the spatial belief sub-symbolic, so occluded regions are never named, inspectable state the planner can target. Reactive, partially-grounded methods [26] avoid probabilistic modelling but cannot plan multi-step information-gathering. Belief-space replanning [11] determinises and replans, as we do, but does not make occlusion a first-class planning entity. The POD tradition [5] reasons about knowledge symbolically and deterministically, but has not been applied to TAMP or to spatial occlusion. This thesis sits at the intersection: a POD task-and-motion planner whose spatial belief is an adaptive, object-centric partition (Boxels)

in which occluded regions and their observed/occluded status are explicit symbolic state. The planner thus reasons about *where* information is missing and composes deliberate look-and-clear actions, without the cost and opacity of a learned probabilistic model.

This position yields three concrete advantages over the alternatives above, each carrying a cost we state plainly. First, *occlusion is first-class, inspectable state*: the planner reasons over named observed-versus-occluded regions a reader can enumerate, rather than over a dense visibility voxel grid feeding a learned statistical effect model, as in TAMPURA's *Find Die* benchmark [7]; the cost is that the visibility signal populating those regions is currently supplied by an oracle rather than a perception module. Second, *uncertainty is handled without probabilities*: knowledge literals record what is known and unknown deterministically, so there is no per-controller probability to estimate and no offline model to learn; the cost is that the planner cannot judge one action as more likely to succeed than another—it minimises plan cost and leans on replanning to recover when an optimistic assumption proves wrong. Third, *no MDP machinery is required*: a lightweight deterministic PDDL layer replaces the learn-a-sparse-MDP-then-solve-with-LAO* stack of POMDP-based TAMP [7], so there is no learned abstraction to mis-estimate; the cost is that optimistic determinise-and-replan is sound only while belief reflects observation, an invariant the bounded give-up on unreachable shadows (Section 6.6) deliberately relaxes.

3.2 Spatial Belief Representation in TAMP

In a typical TAMP cycle the planner (i) reasons symbolically over high-level actions (pick, place, move-to-region), (ii) instantiates them with continuous parameters using a motion planner, (iii) executes the resulting trajectory, observes the environment, and (iv) updates its belief before the next planning step. Every one of these phases depends critically on how we represent spatial belief. While general methods were introduced in Section 2.4, their application in TAMP reveals important limitations.

3.2.1 Limitations of Geometric Discretization for POD-TAMP

Octrees: The efficiency of Octrees makes them a particularly useful tool for 3D mapping. While an Octree framework like OctoMap [18] can represent unobserved space, its primary purpose is to efficiently model occupancy based on noisy sensor measurements. For the task of planning to find hidden objects, it has two key limitations:

1. **Lack of Semantic Structure:** Standard Octree subdivision is purely geometric and based on occupancy variance. It does not create semantically meaningful regions corresponding to, for example, "the volume occluded by object A". The planner would have to infer this semantic meaning from the raw geometric data, which is a complex task in itself.

2. **Object Integrity:** The recursive subdivision of an Octree can arbitrarily split a single physical object into multiple voxels at different tree depths. This complicates reasoning about objects as whole entities, which is essential for manipulation planning (e.g., ”pick up object A,” not ”pick up voxels 1, 2, and 5 of object A”).

3.2.2 Limitations of Static Semantic Abstractions

Sidd's Critical Regions (CRs): The Critical Regions method [29] uses a learned predictor to identify task-relevant regions for each environment from that scene's input, giving the planner a meaningful, non-uniform way to abstract the space. The predictor generalises across environments, but within a given scene the regions are predicted once and then held fixed, so the partition does not adapt when objects move during execution. For example, the region ”behind object A” depends on where A currently is and where the robot is standing—a partition fixed at the start of the episode cannot track that.

4 Approach

This chapter details the methodology for integrating a novel spatial abstraction, Boxels, into a Partially Observable Deterministic (POD) Task and Motion Planning (TAMP) framework built upon PDDLStream.

4.1 Overview

Our methodology links abstract belief representation to continuous geometric reasoning through three components:

1. **Adaptive Semantic Discretization:** A dynamic procedure for partitioning the robot's workspace into a set of meaningful Semantic Boxels, i.e. semantically meaningful subspaces, based on detected objects and their known 3D models.
2. **A Formal POD-TAMP Model:** A formal model that uses these Boxels as the foundation for its state representation, allowing a POD planner to reason about knowledge and uncertainty regarding object locations and unobstructed spaces.
3. **PDDLStream Integration:** An implementation using custom PDDL fluents, actions, and streams that operate on the belief state over Boxels, realizing the formal model in PDDLStream.

4.2 Adaptive Semantic Discretization: Generating Boxels

We build the Boxel abstraction through a process of *adaptive semantic discretization*. This process dynamically generates a set of Semantic Boxels ($\mathcal{BX}_{\text{set}}$) by partitioning the workspace based on detected objects and the occlusions they create. It runs as a Python stage before each call to the planner—and is re-run between actions as the scene changes—rather than as a PDDLStream procedure. Figure 4.1 illustrates the process.

Figure 4.1: Adaptive Semantic Discretization Process. (a) Initial scene with robot viewpoint and objects. (b) Objects are bounded by dedicated Boxels. (c) Occlusion regions are calculated and defined as new Boxels. (d) Remaining free space is recursively partitioned into Free Space Boxels.

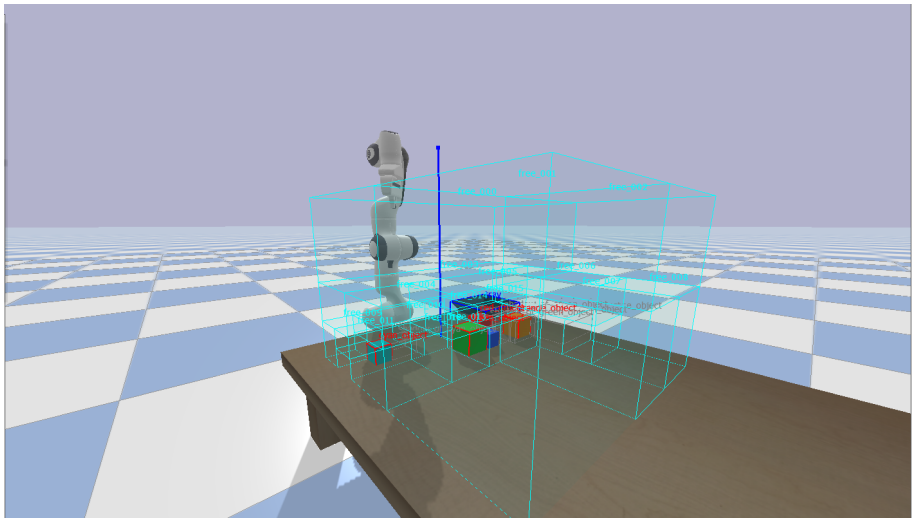
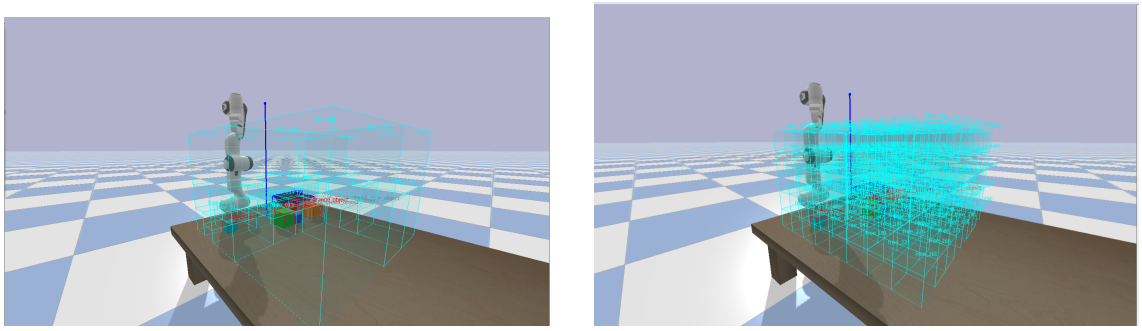


Figure 4.2: The same partition applied to a PyBullet scene. Object bounding cuboids enclose the visible cubes; red wireframe shadows mark the volumes occluded by each object. Free-space cells (not rendered) fill the remaining workspace.

The generation process is as follows:

1. **Object-Centric Bounding:** When an object is detected, we generate a dedicated Boxel: a cuboid that tightly bounds the object's known 3D model at its detected pose. This isolates each known object in its own spatial region. Object detection is provided by an oracle that returns ground-truth object identities and poses from the simulator, which isolates the planning contribution from perception noise; replacing it with a learned, image-based detector is left to future work.
2. **Occlusion-Aware Subdivision:** For each object-bounding Boxel, the space occluded by it from the fixed camera viewpoint is calculated and partitioned into one or more shadow Boxels, split by depth and by any intervening visible obstacles. As the robot acts, the set of shadow Boxels is updated: sensing a region resolves its shadow, and discovering a previously hidden object adds a shadow for that object. A relocated object, however, is not given a new shadow at its destination. Re-creating shadows for moved objects could admit plans that never terminate, so it is deliberately omitted; handling it safely would require additional belief bookkeeping and is left to future work. This explicitly represents regions the robot currently cannot observe, so the planner can target them with information-gathering actions.
3. **Recursive Partitioning:** The space not occupied by object or occlusion Boxels is partitioned recursively, starting from a single Boxel that spans the whole workspace. A Boxel found to be empty is kept as a Free Space Boxel; a Boxel that still overlaps an object or occlusion is split into eight octants, and the procedure repeats on each. Subdivision stops when a region is empty or a minimum Boxel size is reached. The resulting free cells are then merged greedily across shared faces into larger convex Boxels, which reduces their number without changing the represented free volume. This merge is convex-only—two cells are combined only when they share an exactly aligned face—so the partition retains more, smaller free Boxels than a full semantic merge would; this is accepted as sufficient for the object counts evaluated here (see Section 6.6).

This adaptive, object-centric approach contrasts with a uniform voxel grid by focusing discretization effort on task-relevant areas—the objects themselves and the occluded spaces they create—rather than partitioning all space uniformly. Its free-space stage is itself an octree, and a uniform-grid baseline is implemented for comparison. Each generated Boxel is assigned a unique ID and associated geometric data; the completed set of Boxels is supplied to the planner as static facts in its initial state. Figure 4.3 contrasts the two partitions on a matching scene.



(a) Semantic partition: object boxels and free space where the scene is empty.

(b) Uniform-grid baseline: hundreds of cells of fixed size cover the workspace.

Figure 4.3: Adaptive semantic partition (left) versus uniform-grid baseline (right) on a matching scene. The adaptive partition produces a small set of task-relevant cells; the uniform grid covers the whole workspace at the same cell size, producing an order of magnitude more cells.

4.3 Belief over Boxels with Know-If Fluents

The Semantic Boxels generated in the previous section partition the workspace into a finite set of regions; what the robot does not know is which Boxel a hidden object occupies. We represent this uncertainty with the Know-If Fluents introduced in Chapter 2. For each object obj and Boxel $Boxel$, the belief carries the K-literal $K(\text{InBoxel}(obj, Boxel))$ when the robot knows the object is in that Boxel and $K(\neg\,\text{InBoxel}(obj, Boxel))$ when it knows the object is not; the absence of both means the object is possibly there. Because the set of Boxels is finite, the belief over object locations is finite as well, and it is held directly in the planner's symbolic state.

Representing belief this way is what keeps planning tractable. As described in Chapter 2, the K-literal translation compiles a partially observable problem into a classical one. Applied here, it lets an ordinary classical planner reason directly about where objects might be and plan sensing actions to resolve that uncertainty within the same classical search, rather than delegating belief to a separate contingent or belief-space planner. This compile-to-classical reduction is the one introduced by the POD planners CLG and LW1 [1, 5]; we adopt their knowledge-literal translation and solve the result by optimistic determinisation and reactive replanning (Chapter 3).

Knowing which Boxel holds an object is not enough on its own: a manipulation action also needs a reachable grasp and a collision-free trajectory. For this continuous-geometry side we rely on PDDLStream, which extends classical PDDL with *streams*—the lazily-evaluated samplers and tests for continuous parameters described in Chapter 2. Our approach combines the two in a single PDDL domain: Know-If Fluents carry the discrete belief over Boxels, while streams certify the continuous facts—grasps, inverse-kinematics solutions, and motions—

that the manipulation actions require. A symbolic action such as picking a target object therefore mixes both kinds of precondition: a Know-If Fluent stating that the target's Boxel is known, and stream-certified geometric facts stating that the grasp and motion are feasible. The following two sections make this concrete, giving first the formal POD-TAMP model and then its PDDLStream realization.

4.4 Semantic POD-TAMP Model Definition

We formalize our Semantic POD-TAMP system as an explicit state model, adapting the state-space formulation of Geffner and Bonet [12] to the partially observable deterministic setting. The model is a tuple:

$$\mathcal{M} = \langle \mathcal{S}, \mathcal{S}_0, \mathcal{S}_G, \mathcal{A}, f, \mathcal{O} \rangle$$

where:

- $\mathcal{S}$ **(States):** A finite set of abstract states. Each state is a consistent assignment of truth values to a set of atomic propositions $\mathcal{AP}$ built from K-literals (see Chapter 2). These propositions are grounded in the Boxel representation, e.g., $K\big(\text{InBoxel}\big(obj_k, Boxel_j\big)\big)$.
- $\mathcal{S}_0$ **(Initial States):** The set of possible initial abstract states, representing the initial belief b_0.
- $\mathcal{S}_G$ **(Goal States):** The set of goal states to be reached with certainty, specified by a logical formula over $\mathcal{AP}$ (e.g., K(holding(target_obj))).
- $\mathcal{A}$ **(Actions):** A set of abstract actions defined as PDDLStream action schemas that operate on the belief state, such as 'sense' or 'pick'.
- f **(Transition Function):** A deterministic state-transition function $s' = f(a, s)$ that applies an action's add and delete effects to the current state, so that the successor state reflects the updated knowledge.
- $\mathcal{O}$ **(Sensor Model):** The sensor model relates a sensing action to the observation it yields—the target is found in a Boxel, or it is not—and the resulting belief update. In our system this is not realized as an observation function inside the POD planner: the planner's sensing action is optimistic and assumes the target is found, while the true outcome is obtained at execution time and folded into the belief by a Python sense-plan-act loop, which replans whenever the optimistic assumption proves wrong.

The planning problem is to find a sequence of actions from $\mathcal{A}$ that moves the belief state from $b_0 = \mathcal{S}_0$ to a belief state b_g in which every possible state is a goal state (i.e., $b_g \subseteq \mathcal{S}_G$).

4.5 PDDLStream Integration

This section presents the implemented PDDL domain and streams that realize the model on the belief state over Boxels. For readability the listings show the belief-related fragment of the domain, eliding the geometric and stacking predicates that are not central to this discussion.

4.5.1 PDDL Fluents for Belief Representation

The agent's knowledge is represented directly in the PDDL state. The implemented domain is untyped STRIPS, so object categories are themselves predicates, and the belief over object locations is carried by a single Know-If fluent.

```
(:predicates
    ; --- Category predicates (untyped domain) ---
    (Boxel ?x)               ; ?x is a Boxel
    (Obj ?x)                 ; ?x is a manipulable object

    ; --- Boxel classification ---
    (is_shadow ?b)           ; ?b is an occluded region
    (is_object ?b)           ; ?b bounds a physical object
    (is_free_space ?b)       ; ?b is known-empty space

    ; --- Object location: value and knowledge ---
    (obj_at_boxel ?o ?b)     ; object ?o is at Boxel ?b
    (obj_at_boxel_KIF ?o ?b) ; Know-If fluent: it is known whether ?o is at ?b

    ; --- Other knowledge and robot state ---
    (obj_pose_known ?o)      ; ?o has been localized to a Boxel
    (handempty)              ; the gripper is empty
    (holding ?o)             ; the gripper holds ?o
    ; ... further robot, geometry, and stacking predicates
)
```

Listing 4.1: PDDL fluents for belief over Boxels

Section 4.3 describes belief with the two K-literals $K(\text{InBoxel})$ and $K(\neg\,\text{InBoxel})$. The implemented domain realizes the same belief with a single Know-If fluent: `obj_at_boxel_KIF` marks that the location of an object relative to a Boxel is *known*, while `obj_at_boxel` carries the value; the absence of the Know-If fluent means the location is still uncertain. The two encodings are logically equivalent, and the single fluent keeps the grounded state smaller.

4.5.2 The Sensing Action

Information gathering is realized by a single `sense` action, which checks whether a target object is present in a given Boxel.

```
(:action sense
```

```
    :parameters (?o ?region)
    :precondition (and
        (Obj ?o)
        (Boxel ?region)
        (view_clear ?region)                ; line of sight to the region is clear
        (boxel_fits ?o ?region)             ; ?o physically fits inside ?region
        (not (obj_at_boxel_KIF ?o ?region))  ; only sense while the location is
            unknown
    )
    :effect (and
        (obj_at_boxel_KIF ?o ?region)       ; the location of ?o is now known
        (obj_at_boxel ?o ?region)           ; optimistic: assume the object is
            found here
        (obj_pose_known ?o)
        (increase (total-cost) 1)
    )
)
```

Listing 4.2: The implemented sense action

The action is *optimistic*: its effect unconditionally asserts that the object is found in the sensed region. An earlier design used conditional effects that branched on a `found`/`not_found` observation, which would let the planner build a multi-step search ("sense A; if empty, sense B") within one plan. PDDLStream and its classical backend do not support such contingent planning, so the implemented domain commits to the optimistic outcome and relies on *reactive replanning*: when execution reveals that the target is not in the sensed region, the execution loop updates the belief and replans. An empty region is recorded as clear; a region that instead turns out to hold a *non-target* object has that object—and the new region it occludes—added to the partition, re-boxelizing the workspace before the next plan so the newly revealed occluder enters the belief rather than being discarded. With N candidate regions this converges in at most N replanning cycles and is functionally equivalent to a conditional plan executed in sequence; optimistic planning with replanning on failure is a standard pattern for TAMP under partial observability. Figure 4.4 shows the action targeting a shadow region in the simulator, and Figure 4.5 the execution-time moment when the optimistic assumption is rejected and a replan is triggered.

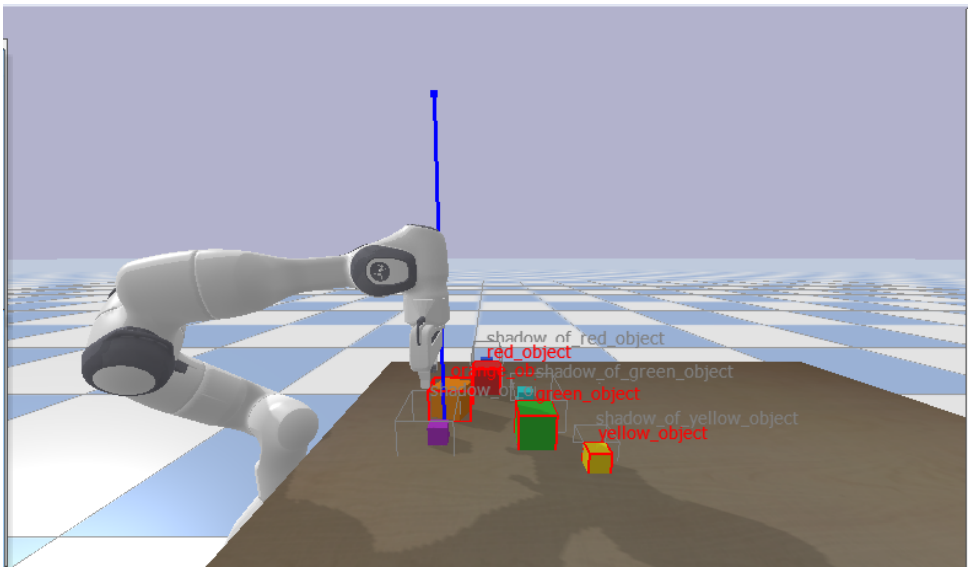


Figure 4.4: The `sense` action targeting a shadow region in PyBullet. The arm has been moved to a sensing configuration over a labelled shadow; the in-simulator overlay labels each object and its shadow region. This is the configuration in which the `sense` action's `(view_clear ?region)` precondition is checked and its optimistic `(obj_at_boxel ?o ?region)` effect is asserted.

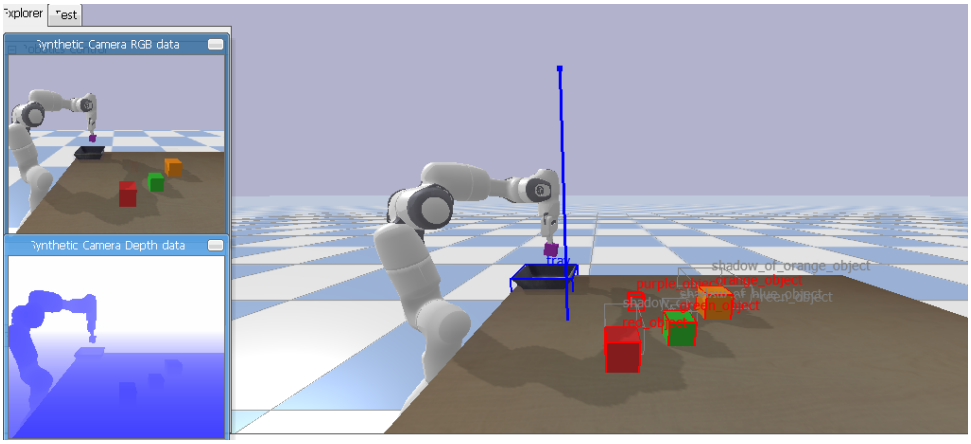


Figure 4.5: The reactive-replan trigger. The action log reads "sense shadow_of_purple_object — target not here": execution has rejected the optimistic add-effect, so the loop marks that shadow empty in the belief and replans, now searching only the remaining shadows.

One bounded exception to this observation-driven belief update concerns regions that cannot be observed at all: if the line of sight to a shadow remains blocked after a fixed number of sensing attempts, that shadow is marked empty without ever being directly observed. This keeps the planner progressing when a region is effectively unreachable, but it is unsound—the region could still hold the target. A sound treatment, which would re-ground the view-blocking facts so the planner first clears the obstruction, is left to future work.

The sense action calls no streams. Because a fixed overhead camera observes the scene, no robot sensing configuration has to be sampled, and the only geometric precondition is the derived predicate `(view_clear ?region)`, which holds when no object blocks the line of sight to the region. Streams instead serve the manipulation actions: the domain declares `sample-grasp`, `plan-motion`, `compute-kin`, and `compute-stack-kin`, so that an action such as `pick` requires a valid grasp, an inverse-kinematics solution, and a collision-free motion, each certified by a stream.

5 Evaluation

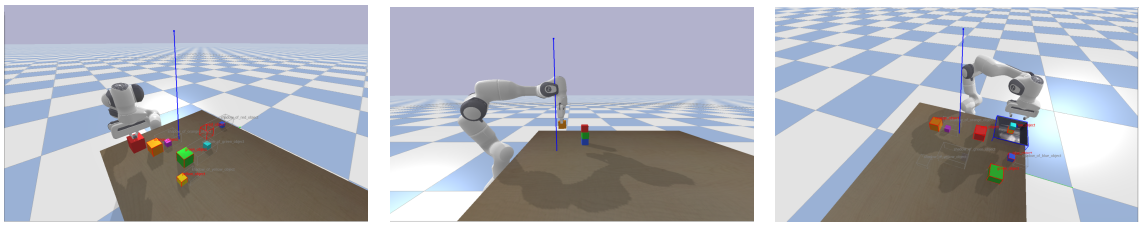
This chapter reports the experimental evaluation of the Boxel-based POD-TAMP framework. Section 5.1 describes the experimental setup as executed. Section 5.2 defines the measured quantities. Section 5.3 defines the variants compared, including the TAMPURA external reference. The results themselves follow in the remaining sections.

5.1 Experimental Setup

Simulator and robot. All experiments were run inside PyBullet [6], with a 7-DOF Franka Emika Panda as the manipulator on a tabletop.

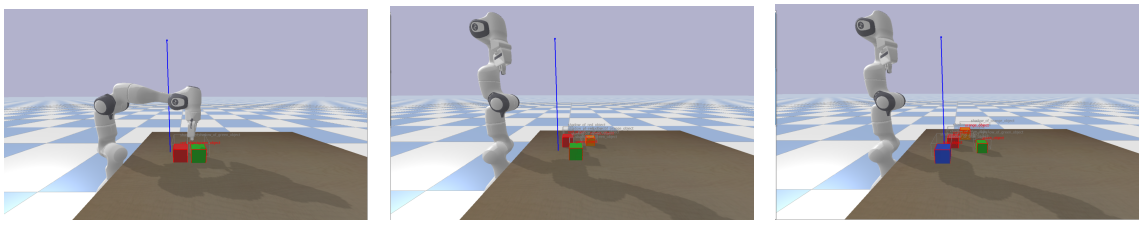
Goals. Three goals were evaluated (Figure 5.1):

- `holding`: the robot must end the episode holding a single target object whose initial pose is hidden behind one of a set of occluders. The number of occluders n_{occ} is the difficulty axis (2, 3, 4; see Figure 5.2).
- `stack`: the robot must build a stack of cubes to a given height. The stack height h is the difficulty axis (2, 3, 4); cube positions are fully observable.
- `find-and-tray-stack`: the robot must locate hidden cubes behind occluders and stack them on a tray. The number of occluders n_{occ} is the difficulty axis (2, 3, 4).



(a) `holding` (b) `stack` (c) `find-and-tray-stack`

Figure 5.1: The three evaluated goals. `holding`: retrieve a single target hidden behind occluders. `stack`: build a tower of cubes to a target height. `find-and-tray-stack`: locate hidden cubes and stack them on a tray.



(a) Low n_{occ} (b) Medium n_{occ} (c) High n_{occ}

Figure 5.2: The occluder-count difficulty axis n_{occ}, shown at low, medium, and high settings. More occluders crowd the target and occlude a larger share of the workspace, so the planner must reason over more shadow Boxels.

Scenes and seeds. Each (goal, variant, difficulty) combination was run on 100 random scenes; seeds are paired across the `semantic` and `semantic+mbs0.05` variants on the random-pairs scene, so their head-to-head comparison is seed-for-seed. A total of ≈2700 cells were executed (3 goals × 3 variants × 3 difficulty levels × 100 seeds, with minor rounding from scheduling).

Perception. Object detection is performed by an oracle that reads ground-truth object poses from the simulator and uses ray-casts from a fixed overhead camera (Figure 5.3) to determine which objects, and which Boxels, are visible. The simulator also renders RGB-D images, but they are not processed inside the planning loop. The modelled uncertainty is occlusion uncertainty—which Boxel a hidden object occupies—resolved at run time by the `sense` action.

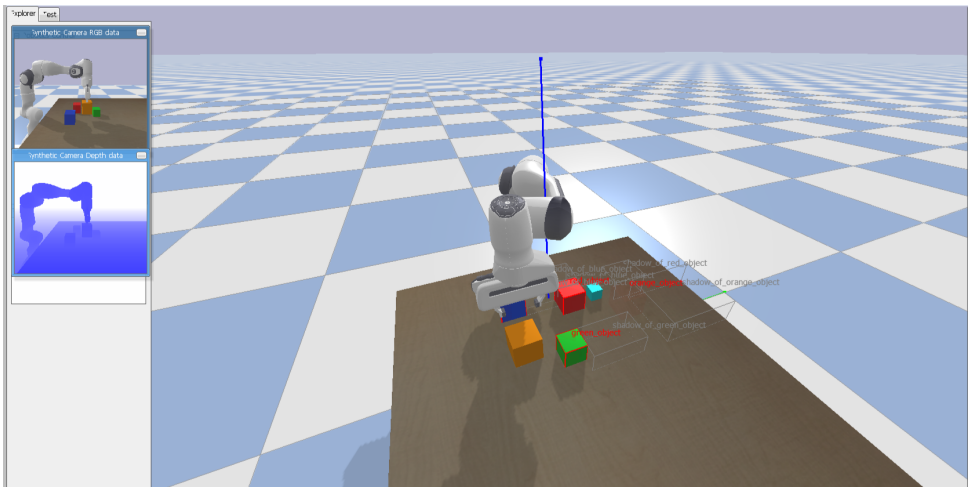


Figure 5.3: An evaluation scene observed by the fixed overhead camera, shown here from an oblique simulator viewpoint. Object detection is an oracle that reads ground-truth object poses from the simulator—objects and their occlusion shadows are labelled in the overlay—rather than processing the camera's rendered RGB-D images. A future learned detector would consume those images in place of the oracle, without changing the planning architecture.

Planning budget. Each PDDLStream call (initial plan or replan) was given a wall-clock budget of 1800 s (30 min); a cell terminates with the `timeout` exit reason if it exhausts this budget. Most successful cells finish in well under a second per call; the budget exists to catch pathological cases rather than as the operating point. The anytime sweep records cumulative solve rate as a function of wall-clock budget, and the curves are essentially flat past ≈200 s.

Software. PyBullet [6]; PDDLStream [10] with the FastDownward classical planner backend; Python 3.10.

5.2 Evaluation Metrics

For each cell we record:

- **Success**: whether the robot reached the goal within the planning budget, without exceeding the per-episode replan limit.
- **Planning time**: total wall-clock time spent inside PDDLStream calls, summed across replans. Wall-clock per cell is effectively equal to planning time; execution, perception, and motion overhead are negligible at the scales evaluated.
- **Replan count**: how many times the planner was re-invoked in the sense-plan-act loop before terminating.
- **Representation size**: the Boxel set decomposed into `OBJECT`, `SHADOW`, and `FREE_SPACE` cells, and the number of grounded facts in PDDLStream's initial state. These two capture the per-cell cost of the partitioning strategy independently of planner choice.
- **Failure mode**: when a cell does not succeed, an exit reason among `planner_failed`, `timeout`, `replan_limit`, `physics_mismatch`, `drop_failed`, `no_summary`, and `all_searched`.

Aggregate means in this chapter are taken either over all n_{cells} or, for quantities that are only defined on cells that produced a snapshot (notably mean initial-state facts and mean replan count), over the subset of cells with non-null values. The two denominators differ by ∼6 % of cells (early timeouts without a recorded state).

5.3 Baselines for Comparison

Three variants of the planner are compared, plus one external reference:

1. **semantic** — the adaptive Boxel partition described in Chapter 4. Free-space cells are placed only where the workspace is empty; their leaf size is auto-set to the largest object's footprint (≈6–9 cm), since finer leaves break placement on the table dimensions used here.

2. **semantic+mbs0.05** — the same adaptive partition but with the free-space leaf-size floor forced to 5 cm (`min_boxel_size = 0.05`). This is intended to test the effect of a finer free-space leaf floor independently of the semantic structure.
3. **uniform** — the adaptive partition is replaced by a uniform voxel grid over the workspace; the object and shadow Boxels are unchanged. The cell size is auto-set to the same largest-object-footprint value, since finer cells likewise break placement. This isolates the contribution of the adaptive free-space partition.
4. **TAMPURA** [7] — a published POMDP-based TAMP framework. Its Table II reports several model-learning/search configurations on a *Partial Observability* task; we compare against TAMPURA's own primary configuration there—Bayes-Optimistic belief updates with LAO* search—which it reports at 57 ± 38 s over 20 trials (Table II of [7], arXiv v2), rather than re-running it; the comparison is architectural — a probabilistic learned-MDP planner solved with LAO* versus our deterministic knowledge-literal planning with replanning — not a hardware benchmark. See Section 6.3 for the full framing.

5.4 Results

Table 5.1 reports the aggregate success rate and mean success-only planning time for each (goal, variant) cell. The semantic and uniform variants differ by an order of magnitude or more in both axes on every goal; `semantic+mbs0.05` sits within seed noise of `semantic` on stack, modestly ahead on holding, and modestly behind on find-and-tray-stack.

goal	variant	n_{cells}	success rate	mean plan time (s)
find-and-tray-stack	semantic	299	39.8 %	28.37
find-and-tray-stack	semantic+mbs0.05	300	33.0 %	21.49
find-and-tray-stack	uniform	300	1.3 %	156.10
holding	semantic	300	42.3 %	7.78
holding	semantic+mbs0.05	300	46.3 %	7.42
holding	uniform	300	33.3 %	50.35
stack	semantic	300	61.3 %	1.23
stack	semantic+mbs0.05	300	61.3 %	1.23
stack	uniform	301	39.2 %	23.84

Table 5.1: Headline success rate and mean planning time per (goal, variant). Planning time is success-only.

5.4.1 Anytime solve rate

Figure 5.4 plots cumulative solve rate against wall-clock budget on a log axis. Three observations stand out. First, the semantic curves rise to their final value within ∼10–30 s on every goal, and the additional budget out to 1800 s buys at most a few extra successes; the operating point is the

first few minutes, not the long tail. Second, on stack the `semantic` and `semantic+mbs0.05` curves overlap exactly — the 5 cm leaf floor changes nothing here. Third, the `uniform` curve is delayed in onset on every goal and tops out lower; on find-and-tray-stack it never gets off the ground (4 solves out of 300).

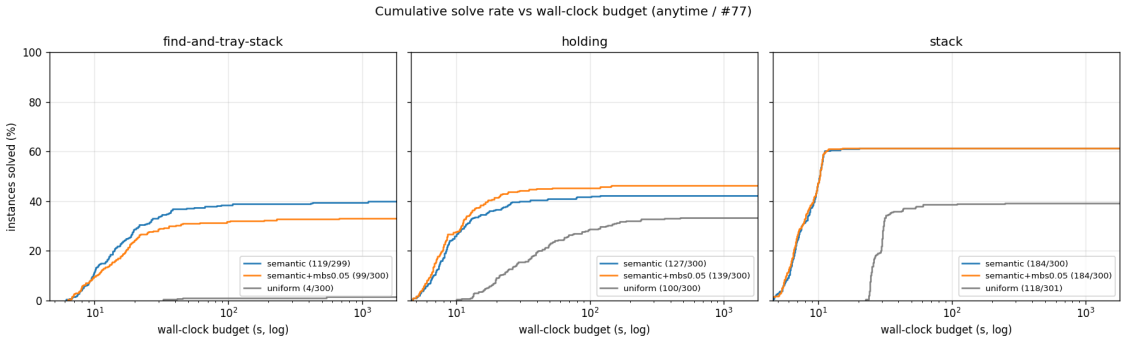


Figure 5.4: Cumulative solve rate vs wall-clock budget (log axis) per (goal, variant). Almost all solves happen within ∼200 s; `uniform` starts later and plateaus lower on every goal.

5.4.2 Success rate and scene difficulty

Figures 5.5 to 5.7 show success rate against the difficulty axis for each goal. On holding (Figure 5.5), success is roughly flat in $n_{\text{occ}} \in \{2, 3, 4\}$: `semantic` stays at 39–47 %, `semantic+mbs0.05` at 44–48 %, `uniform` at 31–36 %. There is no clear scalability degradation in this range, only a ∼10 pp gap between adaptive and uniform partitioning.

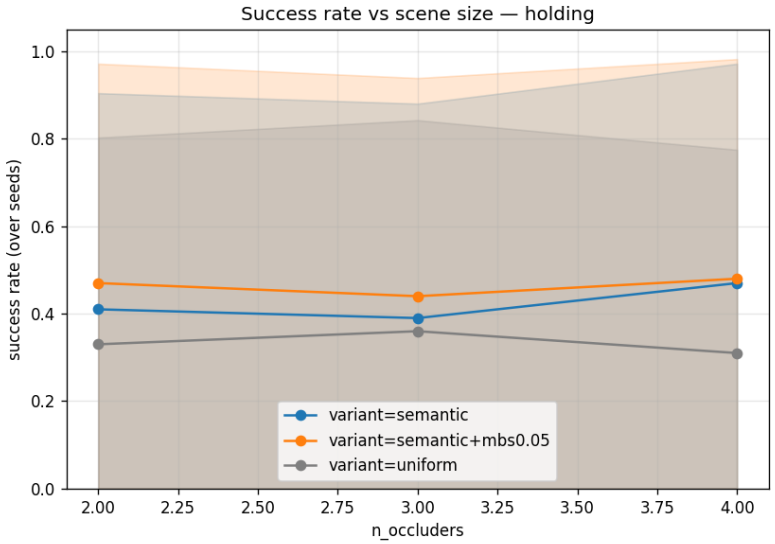


Figure 5.5: Success rate vs n_{occ} on the holding goal. Adaptive variants (semantic, semantic+mbs0.05) are within seed noise of each other; uniform sits ∼10 pp lower.

On stack (Figure 5.6), the difficulty axis is the stack height. All variants solve $h = 2$ trivially (97 %); semantic and semantic+mbs0.05 degrade gracefully to 74 % at $h = 3$ and 13 % at $h = 4$, while uniform collapses from 97 % to 20 % to 1 %. The semantic and semantic+mbs0.05 curves are indistinguishable seed-for-seed.

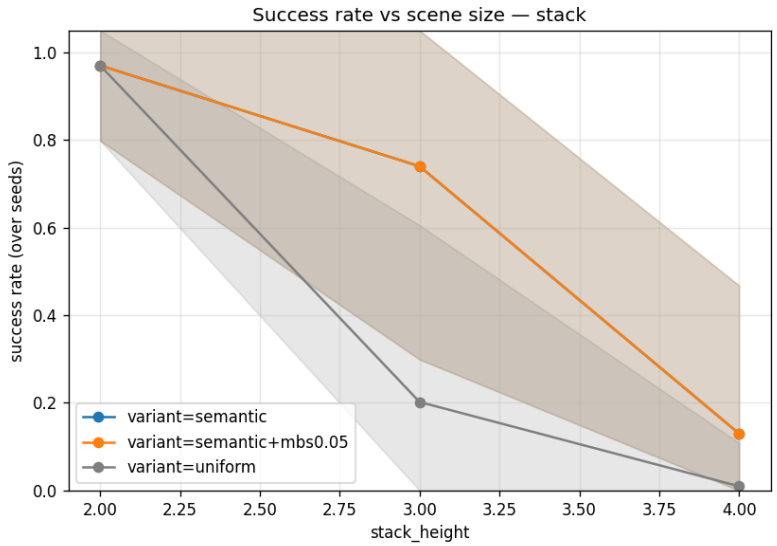


Figure 5.6: Success rate vs stack height h on the stack goal. Adaptive variants degrade gracefully; uniform collapses past $h = 2$.

Find-and-tray-stack (Figure 5.7) is the discriminating goal: `uniform` succeeds on at most 2 out of 100 cells at every difficulty, whereas `semantic` stays flat around 40 %. The `semantic+mbs0.05` variant degrades with n_{occ} (40 % → 31 % → 28 %) on this goal, the only case in the sweep where a finer leaf floor visibly hurts.

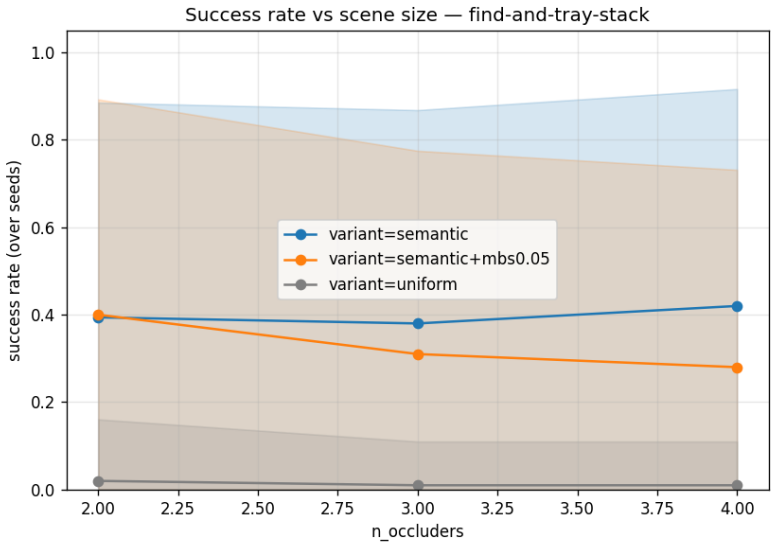


Figure 5.7: Success rate vs n_{occ} on the find-and-tray-stack goal. Uniform fails almost entirely; `semantic+mbs0.05` visibly degrades with n_{occ}, contrary to its intent.

5.4.3 Planning time and scene difficulty

Figure 5.8 shows mean success-only planning time on holding against n_{occ}. The adaptive variants stay below 20 s across the range; `uniform` climbs from 21 s at $n_{\text{occ}} = 2$ to 83 s at $n_{\text{occ}} = 4$, four to five times the adaptive cost at each point. Planning times on stack and find-and-tray-stack follow the same pattern (semantic 1–2 s on stack and ∼20–50 s on FATS; uniform 20–220 s and 25–533 s respectively).

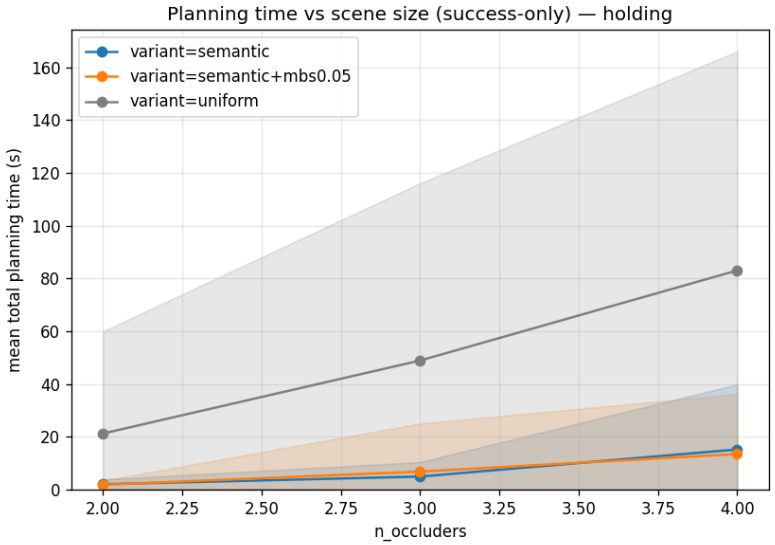


Figure 5.8: Mean success-only planning time vs n_{occ} on the holding goal. Adaptive variants scale gracefully; uniform's cost grows much faster.

5.4.4 Representation compactness

The cost gap traces back to the size of the representation. Figure 5.9 stacks the OBJECT, SHADOW, and FREE_SPACE Boxel counts per variant on holding. The adaptive variants produce ∼25–43 Boxels total; the uniform variant produces ∼300 free-space cells alone. On the stack goal the ratio is steeper (semantic ∼25 vs uniform ∼1340), and the grounded-fact count scales with it: ∼300 facts for the adaptive variants vs 11 000–14 000 for uniform across stack heights. Larger init-state ⇒ larger PDDL grounding ⇒ longer per-call planning time, which is the mechanism behind the timing gap in Figure 5.8.

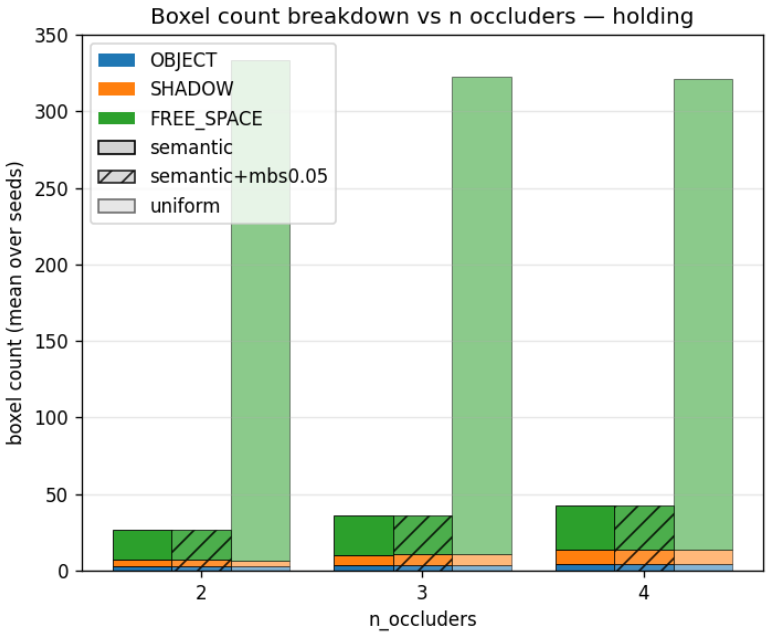


Figure 5.9: Mean Boxel count by category, holding goal. Adaptive variants produce an order of magnitude fewer cells than the uniform baseline.

5.4.5 Comparison with TAMPURA

Figure 5.10 compares per-episode times on our `holding` task — the closest analogue to TAMPURA's hidden-object *Find Die* task (its *Partial Observability* planning times are in Table II of [7]) — with TAMPURA's reported numbers. Two cautions make this a framing exercise rather than a benchmark. First, the spans differ: our per-episode *wall-clock* (mean 13.7 s, success-only) bundles PyBullet execution and replanning, whereas TAMPURA's 57.0 ± 38 s is *planning time only*; the like-for-like quantity is our planning time, a mean 7.8 s. Second, reliability runs the other way — our `semantic` variant succeeds on 42 % of holding episodes, while TAMPURA's discounted return of 0.63 ± 0.30 (Table I, $\gamma = 0.98$, binary terminal reward) implies a success rate of at least 63 %. We are cheaper to plan but less reliable, on a marginally easier goal (*Find Die* also requires returning home; our `holding` goal does not). The comparison is architectural

— a probabilistic learned-MDP planner (LAO*) versus our deterministic knowledge-literal planning with replanning — with no speed or quality winner; Chapter 6 unpacks the framing.

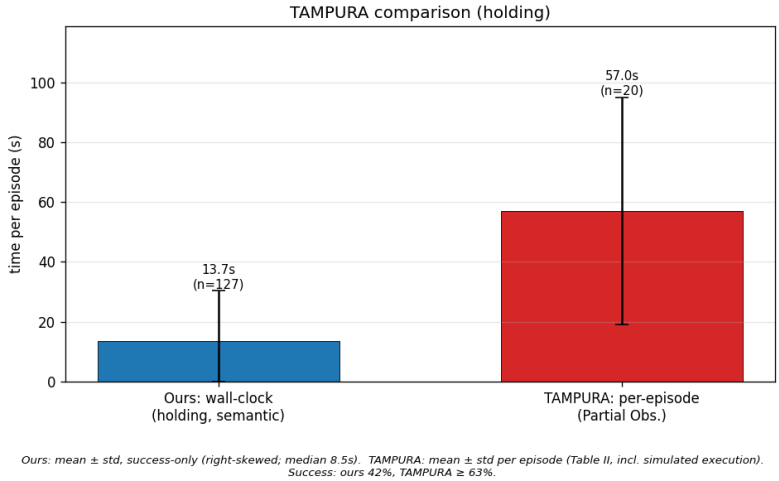


Figure 5.10: Per-episode time on `holding`: our wall-clock and our planning time (both mean ± std, success-only) against TAMPURA's planning-only Partial Observability time (mean ± std, Table II). Wall-clock and planning measure different spans; only our planning time is like-for-like with TAMPURA's. On reliability the order reverses: holding-`semantic` success is 42 % against TAMPURA's inferred ≥ 63 %.

5.4.6 Failure modes

Figure 5.11 stacks the exit reasons across all 9 (goal, variant) cells. `planner_failed` is the dominant failure mode in every condition; `replan_limit` appears predominantly on stack; `timeout` accounts for a small slice of find-and-tray-stack failures. On find-and-tray-stack the `uniform` bar is almost entirely `planner_failed`, consistent with the 1 % success rate.

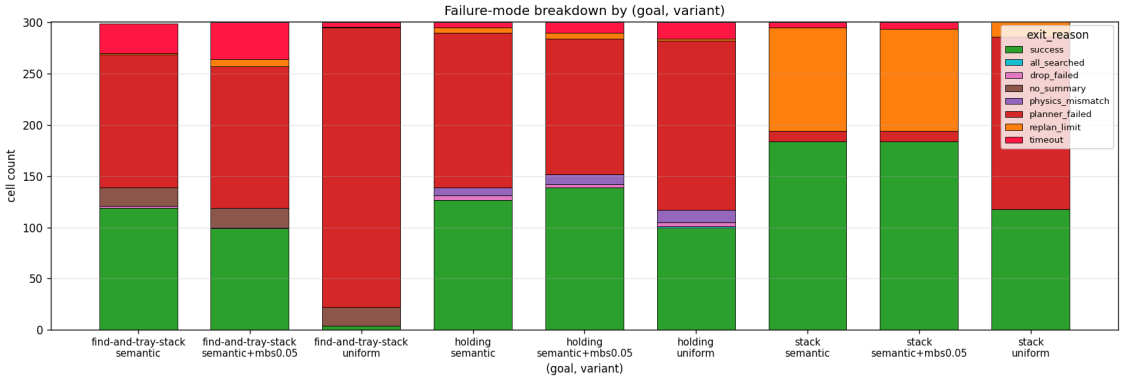


Figure 5.11: Stacked exit-reason breakdown per (goal, variant). `planner_failed` is the dominant failure mode on holding and find-and-tray-stack; on stack the adaptive variants instead fail mainly via `replan_limit` (the second-largest slice after success).

The qualitative readings of these figures, together with their architectural implications and threats to validity, are interpreted in Chapter 6.

6 Discussion

The evaluation in Chapter 5 measures the Boxel-based POD-TAMP framework on three goals (holding, stack, find-and-tray-stack) against a uniform-grid baseline that keeps everything else fixed, and against TAMPURA's published numbers as an external reference. This chapter interprets those results: where the adaptive partition pays off, where finer leaves stop helping, what the TAMPURA comparison does and does not show, and what the failure modes say about the bottleneck.

6.1 Adaptive vs uniform partitioning

The headline finding is the cost gap between the adaptive partition and a uniform free-space grid at the same cell scale. Both partitions use the same auto-set cell size — the largest object's footprint, ≈6–9 cm in our scenes — so the comparison isolates the structural choice rather than the resolution. On holding, the adaptive partition produces 25–43 Boxels per scene; the uniform grid produces ∼320 free-space cells alone (Figure 5.9). On stack the ratio widens to ∼25 vs ∼1340. Larger partitions translate to larger initial-state fact counts (∼300 vs 11 000–14 000 on stack) and longer per-call planning time, which is the mechanism behind Figure 5.8.

What that gap buys is goal-dependent. On holding the uniform baseline still solves about a third of scenes; it pays several times the planning time at every n_{occ} but the mechanism still works. On stack the adaptive variants degrade gracefully from 97 % at $h = 2$ to 13 % at $h = 4$, while uniform collapses from 97 % to 1 %. On find-and-tray-stack, which composes occluded sensing with stacking, uniform is effectively broken (1.3 % success). The reading is that the adaptive partition does not change the planner; it changes how many irrelevant cells the planner has to ground and search over. When the goal needs only a handful of candidate placements — a single hidden target — the overhead of a few hundred free-space cells is tolerable; when the goal requires composing placements across multiple replans, the overhead dominates.

6.2 The 5 cm leaf-floor variant

The `semantic+mbs0.05` variant was included to test whether forcing a 5 cm floor on the free-space leaf size changes outcomes against the default `semantic` configuration, where the leaf floor is the largest object's footprint. It does not: on stack the curves overlap seed-for-seed; on holding the floor is ∼4 pp ahead of `semantic`; on find-and-tray-stack it is ∼7 pp behind, with a visible degradation as n_{occ} grows (Figure 5.7). The mean replan count rises slightly on holding

and find-and-tray-stack relative to `semantic` — consistent with more sensing/replanning cycles without more successes.

The mechanism is that the largest object's footprint (∼6–9 cm in our object set) is already the binding constraint on placement; the 5 cm floor cannot shrink cells past this point and contributes only at the cost of more cells in the partition. This is a characterisation of the regime, not a defect of the method. In scenes whose placements are constrained by sub-5 cm geometry — narrow pockets, dense small-occluder shelves — a finer leaf floor would in principle help. Constructing such a scene and re-running the same comparison is the natural follow-up; it is noted in Section 7.1.

6.3 Architectural comparison with TAMPURA

Figure 5.10 sets our `holding` results against TAMPURA's reported Partial Observability numbers (Table II of [7]); `holding` — find a hidden object and pick it up — is the closest analogue to TAMPURA's *Find Die* task. Read carefully, the comparison cuts two ways. On time, TAMPURA's 57 ± 38 s (Table II) includes executing the selected controllers in simulation, so it is a per-episode figure rather than planner time alone; the like-for-like quantity is therefore our per-episode wall-clock (mean 13.7 s, success-only, which likewise includes PyBullet execution and replanning). On reliability the order runs against us: our holding success rate is 42 %, below the ≥ 63 % implied by TAMPURA's 0.63 ± 0.30 discounted return (Table I). We are no more expensive end-to-end but less reliable, on a marginally easier goal — *Find Die* also requires returning home, which ours does not. Beyond the numbers, the two systems differ in kind.

Both systems sample geometry *online*, inside the planning loop, and both rely on the same determinised, Fast Downward–based classical planner — but they use it for different ends. TAMPURA's model-learning runs at every control step (Algorithm 2 of [7]). From the current belief it optimistically assumes each action succeeds and runs Fast Downward to propose candidate action sequences that reach the goal; it then samples those actions' real outcomes to estimate how they branch under uncertainty, building a *probabilistic* sparse MDP $\bar{\mathcal{B}}_{\text{sparse}}$, and finally solves that MDP for an uncertainty- and risk-aware policy. Fast Downward thus chooses *which goal-reaching action sequences are worth considering*; the MDP solver chooses *what to do once their outcomes are uncertain*, weighing each branch's probability of success. Our planner runs the same kind of determinised search through PDDLStream, but stops there: it reasons over deterministic knowledge literals, executes the chosen plan, and replans when an optimistic assumption proves wrong — there are no transition probabilities to estimate, no probabilistic MDP, and no policy layer on top. The salient difference is that probabilistic layer — learning the MDP and solving it for a policy — not the symbolic search engine underneath.

This framing changes what the time comparison means. It is not a "semantic-Boxels-beats-TAMPURA" claim. The two figures come from different environments — our tabletop scenes

against TAMPURA's — so the times are not measuring the same problem and benchmark neither system. The planners do also differ in kind — ours runs deterministic classical search and replans, whereas TAMPURA learns a probabilistic sparse MDP and solves it for a policy. What the comparison does show is that a deterministic, replanning-based planner reaches the goal on the closest analogous task without ever estimating the learned sparse-MDP abstraction TAMPURA relies on. Both planners represent the belief with symbolic atoms — including epistemic ones, such as whether an object's pose is yet known — so the contrast is not whether knowledge is symbolic but how it is used: we reason over it deterministically and recover by replanning, with no transition probabilities to estimate, at the price of no risk-weighting over uncertain outcomes (Section 6.6).

Beyond *how* each system plans, the comparison highlights *what* each system can reason about. Because our planner holds occlusion as explicit symbolic state, line of sight is a planning condition: the `sense` action requires a clear view of its target region, and the objects that block a view are explicit facts in the planner's state. Those facts also cover candidate placements: a free cell lying on the camera's line of sight to a shadow is flagged as view-blocking, so the planner will not set an object down where it would block a region it still needs to observe. TAMPURA's *Find Die* benchmark—its hidden-object task, the closest analogue to our setting—instead keeps occlusion sub-symbolic: a visibility voxel grid that only weights the success probability of the `look` action, and a placement sampler that draws object poses with no visibility check, so an object can be placed view-blind. Making space plannable in this way is the point of the representation; its present realization is partial—a clear view is a precondition, but *restoring* a blocked view by first moving the obstruction is left to future work (Section 7.1), and the current loop instead bounds itself with a give-up rule (Section 6.6).

6.4 Failure modes

Figure 5.11 stacks the exit reasons across all nine (goal, variant) cells. Three patterns matter.

First, `planner_failed` dominates everywhere. This means the PDDLStream search exhausted its budget without finding a feasible plan — typically because every grasp or motion sample on a candidate Boxel was rejected or the Boxel itself was unreachable. The bottleneck is search, not perception or execution.

Second, `replan_limit` is the second-largest slice on stack and visible on holding `semantic+mbs0.05`. The sense–plan–act loop is bounded by a maximum number of replans per episode; hitting this bound indicates a shadow whose contents the system could not resolve within the loop. This is the failure path of the bounded three-strike give-up discussed in Section 6.6: when a sensing target returns *still-blocked* three times the loop marks the shadow empty and proceeds; if no candidate shadow remains the episode fails with `replan_limit`.

Third, `timeout` is small everywhere except find-and-tray-stack adaptive variants, where it accounts for roughly 10 % of cells. Find-and-tray-stack is a multi-step composition of search and stacking; episodes that nearly succeed but exhaust the per-call budget on a difficult replan show up here. `uniform` on find-and-tray-stack is almost entirely `planner_failed` (or `no_summary`, which records the same fact at a different point in the execution loop), consistent with the 1.3 % success rate.

Read together, the failure-mode breakdown points at search heuristics and the give-up rule as the two paths most likely to lift success rates without changing the representation.

6.5 Threats to validity

The findings above are conditional on the scenes, the budget, and the oracle perception. Five threats merit explicit mention.

Scene family. All measurements are on tabletop scenes with a fixed overhead camera, a Franka Panda, and 6–9 cm cubes. The adaptive-vs-uniform gap is most pronounced in this regime because free-space cells dominate the uniform partition; in scenes with denser objects or smaller workspaces the ratio would narrow.

Oracle perception. Object detection reads ground-truth poses; a learned detector consuming the rendered RGB-D would surface noise, misclassification, and partial-detection failure modes the current numbers do not reflect.

TAMPURA via published numbers. We compare against TAMPURA's Table II rather than re-running their code. Algorithmic, implementation, and parameter differences are bundled into a single delta. Both planners are single-threaded, and TAMPURA's CPU has a marginally higher base clock (2.5 vs 2.0 GHz), so the hardware difference, if anything, slightly favours TAMPURA rather than us. Re-running both systems on a shared scene file would strengthen the comparison.

Resolution regime. The free-space leaf size behaves as a flat knob across the full ∼4× range tested. Below *auto_cell*, forcing a 5 cm floor changes nothing (the `mbs0.05` null finding); above it, coarsening the free-space leaf to 1.5× and 2× *auto_cell* leaves the success rate essentially unchanged — identical seed-for-seed on stack, within seed noise on holding, and only a mild ∼4 pp drop on find-and-tray-stack — while shrinking the free-space partition by 30–40 %. The cause is structural rather than numerical: the merge step combines free-space cells whenever their merged region is convex, so the partition reflects the convex structure of the free space rather than the leaf size itself. *auto_cell* (the largest visible footprint plus a 1 cm margin) is therefore a convenient choice rather than a tuned constant, and semantic's compactness

advantage over `uniform` is a structural property of the adaptive partition, not an artefact of resolution tuning. Neither result generalises to scenes where placements are constrained by sub-5 cm geometry; constructing such a scene is the natural follow-up (Section 7.1).

Bounded give-up. The three-strike give-up on still-blocked shadows is unsound for shadows that are physically unreachable. The run report distinguishes shadows observed empty from shadows given up this way, and the evaluation counts the give-up case as a failure; Section 6.6 expands on the consequence.

The Limitations and Accepted Simplifications below catalogue the design decisions that fix the scope of these results.

6.6 Limitations and Accepted Simplifications

The system evaluated in this thesis embodies a set of deliberate simplifications. Each was adopted to keep the implementation tractable and to isolate the contribution — semantic Boxel discretization and its integration with partially observable deterministic TAMP — from concerns that are orthogonal to it. This subsection consolidates these simplifications so that the empirical claims of Chapter 5 are read with the correct scope.

Perception. The scene contains a single fixed overhead camera that observes the whole table. Its rendered RGB-D images are not processed, however: object detection is performed by an oracle that queries ground-truth object poses directly from the simulator and uses ray-casts to determine which objects are visible from the camera's viewpoint. There is therefore no learned detector and no sensor-noise model. This isolates the planning contribution from perception error — a real perception module consuming the camera images could replace the oracle without altering the planning architecture. Because the camera is fixed, the robot never moves to a sensing configuration; this is sufficient for the tabletop scenes studied here, where every shadow is visible from one viewpoint, but a robot-mounted sensor would be required for occlusions not visible from any fixed vantage point.

Manipulation and execution. Grasping is constraint-based: a held object is rigidly welded to the gripper, so objects cannot slip or be dropped in transit. Only top-down grasps are sampled, at a single fixed height offset. The final approach phase of a pick uses a local inverse-kinematics solver that bypasses the planner's collision checks, on the standard TAMP assumption that the short final motion is obstacle-free if the pre-grasp pose is valid. After a place or stack, execution adds a small hardcoded vertical lift before returning; this lift is invisible to the planner — it produces no PDDL state change and moves only along the gravity axis — and exists solely to give the sampling-based motion planner a non-pathological start configuration for the next

move. Collision events detected during execution are logged but do not trigger an automatic replan.

Spatial representation. Free-space cells are merged only across exactly aligned shared faces; a full semantic merge — matching view-blocking, observability, and ordering — was not implemented, so the partition contains more Boxels than strictly necessary. When an occluder is relocated, its previously computed shadow geometry and the shadow-blocker map are not recomputed at its new position. This rests on the assumption that the world is static apart from the objects the robot itself moves: nothing else shifts, so the only outdated geometry belongs to the relocated object, whose new pose the planner tracks explicitly and replans from. The symbolic interface also uses string identifiers as proxies for geometric volumes — shadows and moved occluders are passed to the planner as named entities rather than as bounding boxes or occupancy grids — bridging continuous geometry to discrete planning at the cost of a representation that is not purely geometric.

Planning. The primary goal studied is `holding`; `stack` and `find-and-tray-stack` are shipped extensions that broaden goal diversity. Enabling stacking roughly doubles per-call planning time, traced to the conditional-effects requirement on the pick action; this regression is accepted rather than optimized away, a mitigation having been considered and dropped as out of scope. A second, smaller planner-search bias concerns the cost model: the domain gives the `stack` action a cost of 2 while every other action costs 1. This is a hand-tuned bias rather than a claim about stacking—because the planner minimises total plan cost and cannot weigh one action's outcome likelihood against another's, the higher cost is what keeps it from substituting a `stack` as a cheap rescue when a `place` onto a free Boxel fails its motion check, forcing it to exhaust placement options first. It is a deterministic stand-in for the probabilistic action selection the POD layer does not perform; `stack` remains available whenever the goal requires it. More fundamentally, when a shadow returns *still-blocked* three times in succession the execution loop marks it as not containing the target and proceeds. The shadow is never directly observed, so for a physically unreachable shadow this violates the invariant that belief should reflect observation. It is accepted to keep the sense–plan–act loop bounded; the run report distinguishes shadows observed empty from shadows given up this way, and the evaluation counts the give-up case as a failure. The principled remedy — re-grounding the planner's view-blocking atoms from current physics — is left to future work (Section 7.1). Figure 6.1 catches this event in the action log.

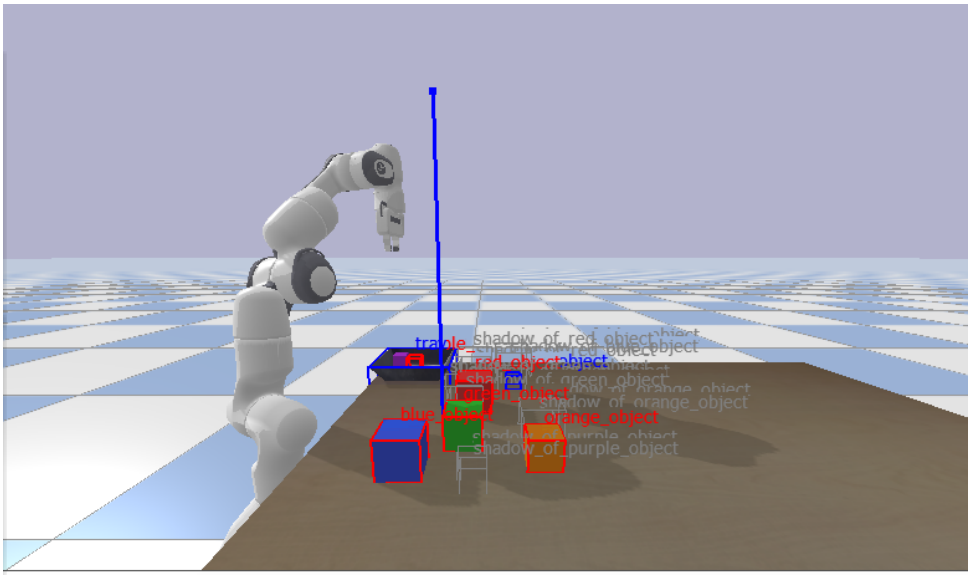


Figure 6.1: The bounded give-up in action. The log line “retry 3/3” marks the third consecutive still-blocked sensing attempt on a shadow; at this point the loop marks the shadow empty and proceeds, even though it was never directly observed. This is the unsound shortcut discussed above.

Parameters. Several geometric constants — grasp height offsets, approach and lift heights, the octree’s minimum leaf size, the camera pose — are tuned for the specific table, robot, and object set used here and would not transfer unchanged to a different setup. Generalizing them, for example by deriving grasp parameters from object geometry, is orthogonal to the core contribution and is noted as future work.

Simplifications disclosed in the approach. Two further simplifications are stated where they are introduced rather than here. The conceptual belief model uses two Know-If literals, $K(p)$ and $K(\neg p)$; the implemented domain collapses these into a single Know-If fluent paired with a value-carrying fluent, an equivalent encoding for the scenarios studied (Section 4.3). The `sense` action is modeled optimistically — it assumes the target is found — with reactive replanning when execution reveals otherwise, because PDDLStream and FastDownward do not support contingent planning (Chapter 4).

7 Conclusion

This thesis presented a novel approach to Task and Motion Planning under partial observability by integrating a semantic spatial abstraction, termed Boxels, with a Partially Observable Deterministic planning framework. At the core of the methodology is adaptive semantic discretization: it partitions the workspace into Boxels, concentrating detail on the objects and occluded regions that matter for the task. By modeling belief states over these Boxels, our system lets a PDDLStream-based TAMP framework plan around uncertain object locations and occlusions.

The Semantic POD-TAMP model formalizes this integration, enabling the planner to generate and execute plans that interleave information-gathering and manipulation actions. The use of K-literals to represent knowledge within the PDDL state allows for a direct and conceptually simple integration with existing POD planners. As demonstrated, this enables the robot to reason about what it knows and where uncertainty lies, and to take actions to resolve that uncertainty in a targeted manner.

This work contributes a representation in which occlusion is first-class symbolic state, letting a TAMP framework handle partial observability without enumerating an exhaustive state space and without the sub-symbolic visibility grid of a POMDP-based TAMP baseline such as TAMPURA's *Find Die* benchmark. The evaluation isolates this representation by ablation—the same planner on the adaptive partition against a uniform grid at the same cell scale—and shows on three tabletop goals that it produces an order of magnitude fewer cells, lowering per-call planning time accordingly and solving find-and-tray-stack scenes that the uniform baseline effectively cannot (39.8 % vs 1.3 % success). The semantic+mbs0.05 variant—forcing a 5 cm floor on the free-space leaf size—is indistinguishable from the default in the regime tested, indicating that placement is bottlenecked on object footprint rather than free-space resolution at these scene scales. Against the published TAMPURA POMDP-based framework, on the `holding` task — the closest analogue to TAMPURA's *Find Die* — our planner plans in a mean 7.8 s against TAMPURA's planning-only 57 s, though it succeeds less often (42 % vs an inferred ≥ 63 %); the comparison is architectural rather than a hardware benchmark, since the difference reflects the planning machinery: TAMPURA learns and solves a probabilistic sparse MDP each step, while our deterministic knowledge-literal planner relies on replanning. The remaining directions for extending this work are outlined below.

7.1 Future Work

The simplifications consolidated in Section 6.6 each suggest a concrete next step. The following directions would extend the system beyond the tabletop scenes evaluated here without altering its core architecture.

Learned perception. The most direct extension is to replace the oracle detector with a learned perception module that operates on the rendered RGB-D images rather than ground-truth simulator poses. Because the oracle is confined behind a narrow detection interface, such a module could be substituted without changing the Boxel discretization or the planning layer, turning the architecture's perception-agnosticism from a claim into a demonstrated property.

Active sensing with a robot-mounted camera. The current fixed overhead camera observes the whole table at once. A robot-mounted sensor would require the planner to reason about *where to look*: a sensing-configuration stream that computes a robot pose from which a target region becomes visible, integrated as an additional precondition on the `sense` action. This is the prerequisite for scenes with occlusions not visible from any single fixed viewpoint.

Discovering objects beyond anticipated shadows. When the robot senses a shadow region, any object hidden there is discovered and added to the representation. The scenes studied here are arranged so that every place an object can hide is one such anticipated shadow region. Extending the approach to objects hidden inside concave geometry — such as a container or cupboard, whose interior is itself a hidden region that may hold further objects and occlusions — would require the partitioning to recurse: re-running over the newly revealed interior and recursively bounding whatever it exposes.

Full semantic free-space merge. Free-space cells are presently merged only across exactly aligned shared faces. A full semantic merge that also matches view-blocking, observability, and back-to-front ordering would yield a coarser, more task-relevant partition with fewer Boxels, reducing the planner's initial-state size.

A PDDL Specification

This appendix lists the complete PDDL domain and stream definitions used by the PDDLStream planner throughout the thesis. The domain (Section A.1) declares the predicates, derived predicates, and the five actions `sense`, `move`, `pick`, `place`, and `stack`. The streams (Section A.2) declare the four samplers that PDDLStream invokes lazily during search: `sample-grasp`, `plan-motion`, `compute-kin`, and `compute-stack-kin`.

A.1 Domain

```
;; ============================================================================
;; Semantic Boxel TAMP Domain - PDDLStream Compatible (Untyped)
;; ============================================================================
;;
;; Models the robot's CAPABILITIES (sense, move, pick, place, stack) rather
;; than any specific scenario. Scenario-specific spatial relationships
;; (e.g., which objects block which regions) are expressed as problem-level
;; init facts using generic predicates like blocks_view_at.
;;
;; Uses derived predicates for visibility: blocks_view and view_clear are
;; computed automatically from object positions — actions only need to
;; update spatial state (obj_at_boxel), not view predicates.
;;
;; Uses Know-If fluents for partial observability.
;; Object types are encoded as predicates: (Boxel ?x), (Obj ?x), etc.

(define (domain boxel-tamp)
  (:requirements :strips :equality :derived-predicates :conditional-effects
      :action-costs)

  (:predicates
    ;; --- Type predicates ---
    (Boxel ?x)
    (Obj ?x)
    (Config ?x)
    (Trajectory ?x)
    (Grasp ?x)

    ;; --- Boxel classification ---
    (is_shadow ?b)           ; Region not directly visible from the camera
    (is_object ?b)           ; Physical object (can be picked, placed, stacked)
    (is_free_space ?b)       ; Known empty space
```

```
32
33      ;; --- Visibility geometry (static) ---
34      (blocks_view_at ?obj ?b ?region) ; When ?obj is at ?b, it blocks view to ?region
35
36      ;; --- Visibility state (derived — DO NOT use in action effects) ---
37      (blocks_view ?obj ?region) ; ?obj currently blocks view to ?region
38      (view_blocked ?region)     ; Some object blocks view to ?region
39      (view_clear ?region)       ; No object blocks view to ?region
40
41      ;; --- Ground truth (actual world state) ---
42      (obj_at_boxel ?o ?b)     ; Object ?o is physically at boxel ?b
43
44      ;; --- Know-If fluent (do we know the value?) ---
45      (obj_at_boxel_KIF ?o ?b) ; We know whether ?o is at ?b (true or false)
46
47      ;; --- Robot state ---
48      (at_config ?q)
49      (handempty)
50      (holding ?o)
51      (obj_pose_known ?o)
52
53      ;; --- Stream certified facts ---
54      (valid_grasp ?o ?g)           ; Grasp ?g valid for object ?o
55      (motion ?q1 ?q2 ?t)           ; Trajectory ?t from ?q1 to ?q2
56      (kin_solution ?o ?b ?g ?q)    ; Config ?q for picking ?o from ?b with ?g
57      (config_for_boxel ?q ?b)      ; Config ?q targets boxel ?b (EE inside ?b)
58      (boxel_fits ?o ?b)            ; Boxel ?b is large enough to contain ?o (place
            dest or sense region)
59      (on_surface ?b)               ; Boxel ?b rests on a support surface (table)
60
61      ;; --- Stacking ---
62      ;; (on ?o ?support) means ?o sits directly on top of ?support.
63      ;; (on_table ?o) is the explicit "?o rests on the table" counterpart.
64      ;; (clear ?o) means nothing is stacked on ?o.
65      (on ?o1 ?o2)
66      (on_table ?o)                 ; ?o sits directly on the table
67      (clear ?o)
68      (is_tray ?o)                  ; ?o is the fixed-base tray support
69      (stack_kin ?o ?on_obj ?g ?q)  ; IK config ?q to place ?o on top of ?on_obj
70    )
71
72    ;; ============================================================================
73    ;; ACTION COSTS: bias planner toward place over stack
74    ;; ============================================================================
75    ;; All actions cost 1 except stack (cost 2).  Without this, the planner
76    ;; treated stack and place as equally cheap and chose stack as a "rescue"
77    ;; whenever motion planning to a particular free boxel failed, instead of
78    ;; trying other free boxels.  Higher stack cost forces the search to
```

```
79  ;; exhaust place destinations before falling back to stack.
80  (:functions (total-cost))
81
82  ;; ============================================================================
83  ;; DERIVED PREDICATES: Visibility from object positions
84  ;; ============================================================================
85  ;; blocks_view_at is a static geometric fact in the init state.
86  ;; blocks_view is derived: true when an object is currently at a position
87  ;; that geometrically blocks a view corridor.
88  ;; view_clear is derived via stratified negation: true when no object
89  ;; blocks the view.
90
91  (:derived (blocks_view ?obj ?region)
92    (exists (?b)
93      (and (obj_at_boxel ?obj ?b)
94           (blocks_view_at ?obj ?b ?region))))
95
96  (:derived (view_blocked ?region)
97    (exists (?obj)
98      (blocks_view ?obj ?region)))
99
100 (:derived (view_clear ?region)
101   (and (Boxel ?region)
102        (is_shadow ?region)
103        (not (view_blocked ?region))))
104
105 ;; ============================================================================
106 ;; SENSE: Observe a region to check for an object
107 ;; ============================================================================
108 ;; Requires clear line of sight to the region.
109 ;; Uses the fixed scene camera — no robot positioning needed.
110 ;;
111 ;; Optimistic effect: assume the target is found.  If execution reveals
112 ;; otherwise, the outer loop marks the shadow empty and replans, which
113 ;; avoids contingent planning that PDDLStream and FastDownward do not
114 ;; support.
115 (:action sense
116   :parameters (?o ?region)
117   :precondition (and
118     (Obj ?o)
119     (Boxel ?region)
120     (view_clear ?region)
121     (boxel_fits ?o ?region)              ; ?o physically fits inside ?region
122     (not (obj_at_boxel_KIF ?o ?region))  ; Only sense if unknown
123   )
124   :effect (and
125     (obj_at_boxel_KIF ?o ?region)        ; Now we know
126     (obj_at_boxel ?o ?region)            ; OPTIMISTIC: assume found
```

```
127       (obj_pose_known ?o)
128       (increase (total-cost) 1)
129     )
130   )
131
132   ;; ============================================================================
133   ;; MOVE: Move robot from one configuration to another
134   ;; ============================================================================
135   ;; ?b is the destination boxel — the stream that produced ?q2 certifies
136   ;; that the end-effector at ?q2 is within boxel ?b.
137   (:action move
138     :parameters (?q1 ?q2 ?b ?t)
139     :precondition (and
140       (Config ?q1)
141       (Config ?q2)
142       (Boxel ?b)
143       (Trajectory ?t)
144       (at_config ?q1)
145       (config_for_boxel ?q2 ?b)
146       (motion ?q1 ?q2 ?t)
147     )
148     :effect (and
149       (at_config ?q2)
150       (not (at_config ?q1))
151       (increase (total-cost) 1)
152     )
153   )
154
155   ;; ============================================================================
156   ;; PICK: Pick up an object from a boxel
157   ;; ============================================================================
158   ;; Must KNOW object is there (KIF=true AND at=true).
159   (:action pick
160     :parameters (?o ?b ?g ?q)
161     :precondition (and
162       (Obj ?o)
163       (Boxel ?b)
164       (Grasp ?g)
165       (Config ?q)
166       (handempty)
167       (at_config ?q)
168       (clear ?o)                    ; can't pick an object with something stacked
              on top
169       (obj_at_boxel_KIF ?o ?b)      ; Must know
170       (obj_at_boxel ?o ?b)          ; Must be there
171       (kin_solution ?o ?b ?g ?q)
172     )
173     :effect (and
```

```
      (holding ?o)
      (not (handempty))
      (not (obj_at_boxel ?o ?b))
      (not (on_table ?o))         ; picking lifts ?o off the table
                                  ; (idempotent if ?o was on a stack instead)
      ;; If ?o was stacked on ?x, picking ?o makes ?x clear again.  For
      ;; holding-goal runs (no (on ...) facts) this grounds to a no-op.
      (forall (?x)
        (when (on ?o ?x)
          (and (not (on ?o ?x)) (clear ?x))))
      (increase (total-cost) 1)
    )
  )

  ;; ============================================================================
  ;; PLACE: Place an object in a boxel
  ;; ============================================================================
  ;; Destination must be free space.
  (:action place
    :parameters (?o ?b ?g ?q)
    :precondition (and
      (Obj ?o)
      (Boxel ?b)
      (Grasp ?g)
      (Config ?q)
      (holding ?o)
      (at_config ?q)
      (is_free_space ?b)
      (on_surface ?b)
      (boxel_fits ?o ?b)
      (kin_solution ?o ?b ?g ?q)
    )
    :effect (and
      (handempty)
      (obj_at_boxel ?o ?b)
      (obj_at_boxel_KIF ?o ?b)
      (on_table ?o)               ; place lands ?o on the table
      (not (holding ?o))
      (not (is_free_space ?b))
      (increase (total-cost) 1)
    )
  )

  ;; ============================================================================
  ;; STACK: Place the held object on top of another object
  ;; ============================================================================
  ;; Mirrors place but the destination is an OBJECT boxel rather than a
  ;; FREE_SPACE boxel.  ``stack_kin`` is certified by compute-stack-kin,
```

```
222   ;; which derives the EE target from ?on_obj's CURRENT top z + the held
223   ;; object's half-height; this is computed each time the planner is
224   ;; invoked so multi-step stacks tolerate per-step settling.
225   ;;
226   ;; (clear ?o) becomes true on the newly-stacked object so it is itself
227   ;; available as a support for a subsequent stack action.
228   (:action stack
229     :parameters (?o ?on_obj ?g ?q)
230     :precondition (and
231       (Obj ?o)
232       (Obj ?on_obj)
233       (Grasp ?g)
234       (Config ?q)
235       (holding ?o)
236       (at_config ?q)
237       (clear ?on_obj)
238       (stack_kin ?o ?on_obj ?g ?q)
239     )
240     :effect (and
241       (handempty)
242       (on ?o ?on_obj)
243       (clear ?o)
244       (obj_at_boxel ?o ?o)        ; OBJECT boxel ID equals object name (a stacked
              object's boxel is itself)
245       (obj_at_boxel_KIF ?o ?o)
246       (not (holding ?o))
247       (not (clear ?on_obj))
248       (increase (total-cost) 2)
249     )
250   )
251 )
```

Listing A.1: The Semantic Boxel TAMP domain.

A.2 Streams

```
1  (define (stream boxel-streams)
2
3    ;; Sample grasp poses for an object
4    (:stream sample-grasp
5      :inputs (?o)
6      :domain (Obj ?o)
7      :outputs (?g)
8      :certified (and (Grasp ?g) (valid_grasp ?o ?g))
9    )
10
11   ;; Plan motion between configurations
12   (:stream plan-motion
```

```
    :inputs (?q1 ?q2)
    :domain (and (Config ?q1) (Config ?q2))
    :outputs (?t)
    :certified (and (Trajectory ?t) (motion ?q1 ?q2 ?t))
  )

  ;; boxel_fits is NOT a stream --- the facts are emitted in the initial
  ;; state.  As a test stream it forced adaptive search to re-evaluate
  ;; every (object, free_boxel) pair across skeletons.

  ;; Compute IK solution for picking
  (:stream compute-kin
    :inputs (?o ?b ?g)
    :domain (and (Obj ?o) (Boxel ?b) (valid_grasp ?o ?g))
    :outputs (?q)
    :certified (and (Config ?q) (kin_solution ?o ?b ?g ?q) (config_for_boxel ?q ?b))
  )

  ;; Compute IK for stacking ?o on top of ?on_obj.
  ;; Reads ?on_obj's CURRENT AABB top + held object half-height to derive
  ;; the EE target.  Certifies config_for_boxel against ?on_obj so the
  ;; preceding move action delivers the arm to the support's OBJECT boxel.
  (:stream compute-stack-kin
    :inputs (?o ?on_obj ?g)
    :domain (and (Obj ?o) (Obj ?on_obj) (valid_grasp ?o ?g))
    :outputs (?q)
    :certified (and (Config ?q) (stack_kin ?o ?on_obj ?g ?q) (config_for_boxel ?q
        ?on_obj))
  )
)
```

Listing A.2: Stream definitions.

List of Acronyms

PDF Probability Density Function

RV Random Variable

List of Symbols

General

$(\cdot)$	arbitrary (scalar) variable
$(\bullet)$	arbitrary vector variable
$\approx$	approximately equal
$\langle \cdot, \cdot, \cdot \rangle$	tuple of multiple variables
$\mathbb{R}$	set of real numbers
$\mathbb{E}[X]$	expected value of a random variable (RV) X
$\mathbb{P}[X = x]$	probability that an RV X takes on the value x
$\frac{\partial f(\bullet)}{\partial x}$	partial derivative of f with respect to x
$\frac{\partial f(\bullet)}{\partial x}\Big\vert_{\boldsymbol{a}}$	partial derivative of f with respect to x evaluated at $\boldsymbol{a}$
$\arg\min_a f(a)$	optimal value of a minimizing the expression $f(a)$
$\arg\max_a f(a)$	optimal value of a maximizing the expression $f(a)$
π	mathematical constant, $\pi \approx 3.142$
$\mathcal{N}(\mu, \sigma)$	Gaussian normal distribution with mean μ and standard deviation σ

Markov Decision Process

t	time step index
$\mathcal{S}$	set of all states
$\mathcal{A}$	set of all actions
$\mathcal{R}$	set of all possible rewards
s, s^t	state at time t, realization of RV S^t
s', s^{t+1}	next state at time $t+1$, realization of RV S^{t+1}
a, a^t	action at time t, realization of RV A^t
a', a^{t+1}	next action at time $t+1$, realization of RV A^{t+1}
r, r^{t+1}	reward at time $t+1$, realization of RV R^{t+1}
S^t	state at time t
A^t	action at time t
R^t	reward at time t
$p(s', r \mid s, a)$	dynamics function specifying a conditional probability density function (PDF) for the next state s' and reward r given the current state s and action a

List of Figures

List of Tables

List of Listings

List of References

[1] A. Albore, H. Palacios, and H. Geffner, "A translation-based approach to contingent planning.," in *IJCAI*, vol. 9, 2009, pp. 1623–1628.

[2] Y. Bai, Z. Wang, Y. Liu, K. Luo, Y. Wen, M. Dai, W. Chen, Z. Chen, L. Liu, G. Li, and L. Lin, "Learning to see and act: Task-aware virtual view exploration for robotic manipulation," *arXiv preprint arXiv:2508.05186*, 2025.

[3] W. Bejjani, W. C. Agboh, M. R. Dogar, and M. Leonetti, "Occlusion-aware search for object retrieval in clutter," in *2021 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, IEEE, 2021, pp. 4678–4685.

[4] B. Bonet and H. Geffner, "Planning under partial observability by classical replanning: Theory and experiments," in *Proceedings of the 22nd International Joint Conference on Artificial Intelligence (IJCAI)*, 2011, pp. 1936–1941.

[5] B. Bonet and H. Geffner, "Flexible and scalable partially observable planning with linear translations," in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 28, 2014, pp. 2235–2241.

[6] E. Coumans and Y. Bai, *PyBullet, a Python module for physics simulation for games, robotics and machine learning*, `http://pybullet.org`, 2021.

[7] A. Curtis, G. Matheos, N. Gothoskar, V. Mansinghka, J. Tenenbaum, T. Lozano-Pérez, and L. P. Kaelbling, "Partially observable task and motion planning with uncertainty and risk awareness," *arXiv preprint arXiv:2403.10454v2*, 2024.

[8] R. E. Fikes and N. J. Nilsson, "Strips: A new approach to the application of theorem proving to problem solving," *Artificial intelligence*, vol. 2, no. 3-4, pp. 189–208, 1971.

[9] C. R. Garrett, R. Chitnis, R. Holladay, B. Kim, T. Silver, L. P. Kaelbling, and T. Lozano-Pérez, "Integrated task and motion planning," *Annual review of control, robotics, and autonomous systems*, vol. 4, no. 1, pp. 265–293, 2021.

[10] C. R. Garrett, T. Lozano-Pérez, and L. P. Kaelbling, *PDDLStream: Integrating symbolic planners and blackbox samplers via optimistic adaptive planning*, arXiv:1802.08705. Also in Proc. of ICAPS 2020, 2018. arXiv: `1802.08705`.

[11] C. R. Garrett, C. Paxton, T. Lozano-Pérez, L. P. Kaelbling, and D. Fox, "Online replanning in belief space for partially observable task and motion problems," in *IEEE International Conference on Robotics and Automation (ICRA)*, 2020, pp. 5678–5684.

[12] H. Geffner and B. Bonet, *A concise introduction to models and methods for automated planning*. Morgan & Claypool Publishers, 2013.

[13] M. Ghallab, D. Nau, and P. Traverso, *Automated Planning: theory and practice*. Morgan Kaufmann, 2004.

[14] N. Gothoskar, M. Ghavami, E. Li, A. Curtis, M. Noseworthy, K. Chung, B. Patton, W. T. Freeman, J. B. Tenenbaum, M. Klukas, and V. K. Mansinghka, *Bayes3D: Fast learning and inference in structured generative models of 3d objects and scenes*, arXiv:2312.08715, 2023. arXiv: 2312.08715.

[15] D. Hadfield-Menell, E. Groshev, R. Chitnis, and P. Abbeel, "Modular task and motion planning in belief space," in *2015 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, IEEE, 2015, pp. 4991–4998.

[16] E. A. Hansen and S. Zilberstein, "Lao*: A heuristic search algorithm that finds solutions with loops," *Artificial Intelligence*, vol. 129, no. 1-2, pp. 35–62, 2001.

[17] T. Hofmann and H. Geffner, "Learning generalized policies for fully observable non-deterministic planning domains," *arXiv preprint arXiv:2404.02499*, 2024.

[18] A. Hornung, K. M. Wurm, M. Bennewitz, C. Stachniss, and W. Burgard, "Octomap: An efficient probabilistic 3d mapping framework based on octrees," *Autonomous robots*, vol. 34, no. 3, pp. 189–206, 2013.

[19] L. P. Kaelbling, M. L. Littman, and A. R. Cassandra, "Planning and acting in partially observable stochastic domains," *Artificial intelligence*, vol. 101, no. 1-2, pp. 99–134, 1998.

[20] L. P. Kaelbling and T. Lozano-Pérez, "Integrated task and motion planning in belief space," *The International Journal of Robotics Research*, vol. 32, no. 9-10, pp. 1194–1227, 2013.

[21] Y. Kim, R. Arora, R. Martín-Martín, P. Stone, B. Abbatematteo, and Y. Sung, "Large-language-model-guided state estimation for partially observable task and motion planning," *arXiv preprint arXiv:2603.03704*, 2026.

[22] Y. Ma, Y. Yuan, S. Wu, and H. Yuan, "A task and motion planning framework for partially observable household manipulation scenes," *Advanced Intelligent Systems*, vol. 7, no. 12, p. 2 400 897, 2025.

[23] D. McDermott, M. Ghallab, A. Howe, C. Knoblock, A. Ram, M. Veloso, D. Weld, and D. Wilkins, "PDDL – the planning domain definition language," AIPS-98 Planning Competition Committee, Tech. Rep. CVC TR-98-003, 1998.

[24] D. Molina, K. Kumar, and S. Srivastava, "Learn and link: Learning critical regions for efficient planning," in *2020 IEEE International Conference on Robotics and Automation (ICRA)*, 2020, pp. 10 605–10 611.

[25] C. Muise, V. Belle, and S. A. McIlraith, "Computing contingent plans via fully observable non-deterministic planning," in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 28, 2014, pp. 2322–2329.

[26] T. Pan, R. Shome, and L. E. Kavraki, "Task and motion planning for execution in the real," *IEEE Transactions on Robotics*, vol. 40, pp. 3356–3371, 2024.

[27] G. Riegler, A. Osman Ulusoy, and A. Geiger, "Octnet: Learning deep 3d representations at high resolutions," in *Proceedings of the IEEE conference on computer vision and pattern recognition*, 2017, pp. 6620–6629.

[28] M. S. Saleem, R. Veerapaneni, and M. Likhachev, "A pomdp-based hierarchical planning framework for manipulation under pose uncertainty," *arXiv preprint arXiv:2409.18775*, 2024.

[29] N. Shah and S. Srivastava, "Using deep learning to bootstrap abstractions for hierarchical robot planning," in *Proc. of the 21st International Conference on Autonomous Agents and Multiagent Systems (AAMAS '22)*, 2022, pp. 1183–1191.

[30] L. Zhao, W. McClinton, A. Curtis, N. Kumar, T. Silver, L. P. Kaelbling, and L. L. Wong, "Seeing is believing: Belief-space planning with foundation models as uncertainty estimators," *arXiv preprint arXiv:2504.03245*, 2025.